Q A L A

Universal Solution Factory Operating System

Design & Architecture Document

v1.0 | March 2026 | Confidential

Field	Value
Document Type	Unified Design & Architecture Document
Version	1.0
Classification	Confidential — Not for Distribution
Status	Final Draft for Review
Date	March 2026
Scope	Full Platform — All Planes, Services & Subsystems
Authors	Qala Platform Engineering & Architecture
Pages	100+ pages
Source Documents	Platform Concept v1 · SDD v1 · SDD v2 · LLD v1 · RFC v2 · PRD v1/v2 · Requirements v1/v2 · Workflows v2 · UI/UX Spec v1 · Business Plan v1
Audience	Engineering · Architecture · Product · Security · Leadership · Investors

This document synthesises and reconciles all prior Qala design artefacts into a single authoritative reference.

Table of Contents

1 Executive Summary

Qala is a Universal Solution Factory Operating System — a single governed platform for the end-to-end creation, management, and lifecycle governance of solutions across every domain and scale. Whether the solution is a software microservice, a pharmaceutical formulation, a financial model, an agricultural protocol, a legal product, or a tax rule set — Qala governs it through a consistent, structured, and intelligence-augmented lifecycle.

CORE THESIS	Every person or organisation that builds something of value deserves world-class delivery infrastructure. Qala makes governed, reproducible, AI-augmented solution lifecycle management accessible to everyone — from a solo developer to a global enterprise.

The software and technology industry spends an estimated 30–50% of engineering effort managing the infrastructure required to build, deliver, and govern solutions — not on the solutions themselves. Qala eliminates this waste by providing a single, opinionated, extensible operating system that spans the full lifecycle: from first idea to production deployment to retirement.

1.1 The Seven Core Problems Qala Solves

Dimension	The Problem Today	Qala's Solution	Success Metric
Build Reproducibility	"Works on my machine" — non-deterministic environments causing CI failures and release delays	Hermetic SDEs with digest-pinned environments; Environment-as-Code	Zero environment-induced CI failures
Security	Manual, late, and reactive security — vulnerabilities discovered after release	SAST/SCA/DAST + SEM embedded at every layer of the platform	95%+ vulnerabilities caught in development phase
Environment Drift	Silent divergence between developer environments, CI, and production	Environment-as-code + continuous drift detection and alerting	All drift detected within 15 minutes of occurrence
Knowledge Silos	Institutional knowledge lives in individuals' heads; lost when teams change	Versioned artefacts + AI semantic search across all solution content	New developer productive in less than 1 day
Governance	Manual, incomplete audit trails; compliance prep takes weeks	Immutable audit trail from day one across every entity and action	Compliance reports generated in under 4 hours
AI Integration	Siloed AI tools that are not integrated into delivery workflows	AI Agent woven through every platform domain — advisory, not blocking	30%+ reduction in defect escape rate
Toolchain Fragmentation	30–50% of effort spent on toolchain management, not value delivery	Unified Solution Factory model with standardised toolchain governance	40–60% reduction in toolchain overhead

1.2 Key Platform Metrics

Metric	Target / Value
Active SDEs at 12 months	10,000 target
SDE provisioning time	< 10 minutes from approval to first build

Metric	Target / Value
API P99 latency (read operations)	< 80ms under nominal load (1,000 RPS per tenant)
API P99 latency (write / command)	< 200ms for command acceptance (event published)
Platform uptime (SaaS Standard)	99.9% monthly
Platform uptime (SaaS Enterprise)	99.95% monthly
Event processing throughput	> 100,000 domain events per second platform-wide
Security scan detection time	< 90 seconds (automated SEM)
Rollback time (Blue/Green deployment)	< 60 seconds
Compliance audit preparation time	< 4 hours (vs. 4–12 weeks industry average)
Recovery Point Objective (RPO)	< 1 minute (event log replication lag)
Recovery Time Objective (RTO)	< 15 minutes (full platform recovery)
Tenant onboarding time	< 2 minutes from signup to first usable SDE
Storage durability (artefact storage)	99.999999999% (11 nines)

1.3 Platform Scope

Qala is not a developer tool, a PLM system, or a compliance platform in isolation. It is the operating system for solution factories of every kind. The platform serves organisations across the following domains:

Domain	Example Solutions	Scale
Personal / Hobby	Home automation, personal finance tracker, self-hosted media server, personal knowledge base	Single user
Small Team / Startup	SaaS product, mobile app, API service, internal operations tool, design system	2–50 users
Professional Services	Consulting methodology, managed IT service, legal product, tax compliance workflow	1–500 users
Enterprise Software	ERP implementation, financial model, supply chain protocol, regulatory system	500–100,000+ users
Physical Goods / CPG	Formulation management, batch release, ingredient specification governance	Regulated manufacturing
Financial Solutions	Quantitative models, investment strategies, risk frameworks, tax rule sets	Financial institutions
Agricultural / AgriTech	Crop management systems, IoT firmware, agronomic rule sets, field trial management	Farms to agribusiness
Research / Academic	Experimental frameworks, dataset governance, computational model versioning	Universities to pharma R&D
Platform / Ecosystem	Software platform, open-source framework, developer SDK, marketplace	Millions of users

1.4 Document Structure

This document is structured as follows: Chapter 2 covers the platform philosophy and binding architectural principles. Chapter 3 defines the core entity model and hierarchy. Chapters 4–6 detail the Solution Factory, SDE, and Solution lifecycle. Chapter 7 covers the Change Control system. Chapter 8 addresses the technology stack and service topology. Chapters 9–12

cover data architecture, security, AI intelligence, and quality management. Chapters 13–15 address deployment, integrations, and domain extensions. Chapter 16 covers the UI/UX design system.

2 Platform Philosophy & Architectural Principles

Qala is architected around a small number of binding principles that govern every design decision. Any component, service, or feature that violates a principle requires explicit architectural justification through the RFC process.

2.1 The Root Factory Principle

ARCHITECTURAL REALITY	Qala is itself a Solution Factory — the Root Solution Factory from which all other factories, environments, models, and solutions descend. Every platform update goes through a Change Control Request. Every release is governed. Every architectural decision is recorded as an ADR in the Qala root factory. The platform is self-describing and self-governing.

2.2 The Ten Binding Architectural Principles

ID	Principle	Binding Constraint	Implementation
P1	Event Sourcing	All state changes are derived from an immutable ordered log of domain events. The event log is the system of record. Projections are derived.	Event log is the authoritative source; all read models are projections rebuilt from events
P2	CQRS	Command and query paths are separated at the service level. No synchronous reads in command handlers.	Commands go to the event store; queries go to read-optimised projections in PostgreSQL + Redis
P3	Immutable Artefacts	Released artefacts, published versions, and audit events are never modified or deleted.	Rollback = activating a previous version, never modifying history. Soft-delete only for drafts.
P4	Tenant Isolation	All data, events, secrets, and compute resources are isolated per tenant. Cross-tenant access is impossible by construction.	Row-level security + tenant-scoped encryption keys. Verified at every storage layer.
P5	Composable Deployment	Every SDE is independently deployable as a self-contained unit. Platform runtime is decoupled from SDE runtime.	SDEs run as Kubernetes-native workloads. Each SDE gets its own namespace.
P6	Domain-Agnostic Core	Core platform services are domain-agnostic. All domain-specific logic lives in Domain Packs.	Core functions correctly with zero Domain Packs installed. Domain packs are independently deployable.
P7	API-First	Every platform capability is exposed through a versioned, documented API before being surfaced in any UI.	UI is a consumer of the API, not a privileged client. API versioning policy: no breaking changes without major version bump.
P8	Zero-Trust Security	All service-to-service communication is mutually authenticated (mTLS). No implicit trust. Every request carries a verifiable identity.	Service mesh (Istio) enforces mTLS. Network-level segmentation is not treated as a security control.
P9	Observability by Default	Every service emits structured logs, OpenTelemetry traces, and Prometheus metrics.	No operational state that cannot be observed without code changes. All services are observable from deployment.
P10	Graceful Degradation	Every service defines its degradation behaviour. No single-service failure causes total platform unavailability.	Circuit breakers, fallback responses, and partial-availability modes defined for every service.

2.3 Platform Design Philosophy

Principle	Description	Implementation Note
Universal by Design	One platform for every solution type — personal to enterprise, software to physical, digital to regulatory	No assumptions about domain; all domain specificity lives in Domain Packs
Hierarchical Composition	Factories produce factories; environments compose environments; solutions nest inside solutions	Hierarchy is unlimited in depth and breadth
Governance as Foundation	Change control, version management, audit trails, and lifecycle states are the substrate of every entity	Not optional features — core to every entity regardless of domain
Distributable & Deployable	SDEs are packaged, versioned, and distributable to any target environment	Local, cloud, edge, or offline — same SDE manifest works everywhere
Composable Everything	Every SDE, model, playbook, and tool is composable from reusable components	No lock-in; no silos; every entity can be assembled from standard building blocks
Root Factory Principle	Qala itself is a solution factory — the platform can produce instances of itself	Tiered, hierarchical, self-similar at every level of scale

2.4 Six-Plane Architecture Model

The Qala platform is organised into six logical planes. Each plane has a defined language, responsibility domain, and deployment boundary.

Plane	Language	Responsibilities
Kernel Plane	Rust	Platform integrity, service registry, health monitoring, event aggregation, cross-domain solution graph engine, platform self-governance
Control Plane	Go	SDE Management, Solution Factory, Identity, Notifications, Change Control (CCR/CCB), Bug Tracking, Release Management, Universal Solution Lifecycle State Machine
Execution Plane	Go / Scala	CI/CD Engine, Test Management, Benchmarking, Hermetic Build, Artefact Registry, Distribution Engine. Go for orchestration; Scala/Akka for reactive streaming pipelines
Intelligence Plane	Rust	Universal AI Agent (domain-aware), Security Event Monitor (SEM), Vulnerability Management, Privacy Management. Domain-specific intelligence models hot-swappable per solution type
Platform Operations	Go / Terraform	Environment Management, Tool Registry, Vendor Integration, Backup and Recovery, Audit and Compliance Reporting
Domain Extension Plane	Go + YAML DSL	Vertical-specific solution type schemas, compliance framework adapters, physical world integration bridges, domain workflow templates, industry vocabulary overlays, role-specific UX personas

DOMAIN PACKS	Domain Packs are the primary extension mechanism. A Domain Pack for CPG defines: the Good/Product/ProductLine solution type schemas; the formulation CCR workflow template; GMP and REACH compliance evidence mappings; ERP integration bridge configuration; and the CPG-specific portal vocabulary. Domain Packs are versioned, independently deployable, and community-contributable.

3 Core Entity Model & Hierarchy

Every entity in Qala is either a solution or produces solutions. The platform is structured as a strict containment hierarchy with unlimited depth. This chapter defines the canonical entity model that every subsystem must conform to.

3.1 Complete Entity Hierarchy

HIERARCHY

QALA PLATFORM (Root Solution Factory) └── Solution Factory ├── Child Solution Factory (unlimited nesting) └── Solution Development Environment (SDE) ├── Solution Model │ └── Solutions (instances of the model) │ ├── Playbook (blueprint & design) │ ├── Solution Book (all docs & content) │ ├── Artefacts (in Artefact Repository) │ ├── Change Control Requests (CCRs) │ └── Releases ├── Toolchain (Toolkits → Toolsets → Tools) ├── Repositories (Artefact | Asset | Capital | Resource) ├── Content Management System (CMS) └── Communications & Networking Module

3.2 Entity Summary Table

Entity	Layer	Produced By	Produces	Governed By
Qala Platform	Root	N/A (the root)	All entity types	Qala platform governance
Solution Factory	Organizational	Qala Platform or Parent Factory	Child Factories, SDEs	Factory policies, RBAC
Solution Dev Env (SDE)	Workspace	Solution Factory	Solutions, Artefacts, Content	SDE config, factory policies
Solution Model	Schema/Blueprint	Factory Admin / Domain Team	Solution instances	Model versioning, domain pack
Domain Pack	Schema Extension	Qala or custom authors	Domain-specific field extensions	Pack versioning, factory approval
Solution	Output	SDE (developer action)	Artefacts, Releases, Change history	CCR, lifecycle state machine, model
Playbook	Documentation	SDE member or AI Agent	Design decisions, ADRs, blueprints	Solution versioning, CMS
Solution Book	Documentation	Auto-generated + SDE members	Comprehensive solution documentation	Solution versioning, CMS, access policy
CCR	Governance	Any authorized user	Approved changes, compliance evidence	Risk routing, approval chain policy
Release	Deployment	Release Manager	Deployed solution, release artefacts	Release policy, quality gates
Artefact	Output	SDE build/toolchain	Deployable packages, evidence records	Artefact repository policies
Toolchain	Infrastructure	Qala Marketplace or custom	Built artefacts, test results	Toolchain versioning, SDE config
AI Agent	Intelligence	Qala platform (system)	Recommendations, risk scores, insights	AI policy, privacy settings
Audit Event	Compliance	Every governed action (automatic)	Compliance evidence packages	Audit vault (immutable)

3.3 The Seven-Level Solution Structure Model

Qala enforces a strict seven-level hierarchy that governs how every solution is decomposed and described. This hierarchy applies universally regardless of domain.

Level	Element	Definition	Example (Software)	Example (Physical)
1	System	Top-level organisational unit grouping applications that fulfil a coherent purpose	Banking Platform	Pharmaceutical Product Suite
2	Application	Deployable unit of functionality residing within a System	Core Banking API	Drug Formulation v2.1
3	Process	Discrete unit of work executed within an Application	Transaction Processing	Granulation Process
4	Component	Bounded, reusable building block within a Process	Auth Module	Excipient Blend Component
5	Interface	Contract surface of a Component; declares Imports and Exports	REST Endpoint Contract	Regulatory Submission Interface
6	Message	Unit of communication flowing through an Interface (Event or State)	Transaction Event	Batch Release State
7	Data Structure	Schema of a Message payload, composed of typed fields	TransactionPayload{}	BatchRecord{}

3.4 Six First-Class Solution Types

Solution Type	Description	Examples
Application	A software application delivered to end-users or integrated systems	Web apps, mobile apps, APIs, microservices, firmware
System	A coordinated collection of applications serving a unified purpose	Banking platform, ERP system, logistics OS
Good	A tangible, physical, or digital deliverable produced by the factory	Physical product, pharmaceutical formulation, hardware component
Product	A commercially packaged, versioned, and distributable artefact	SaaS product, packaged software, drug product
Service	An ongoing capability delivered to consumers on a continuous basis	Managed IT service, consulting service, subscription service
Platform	A foundation on which other solutions are built and operated	Developer platform, marketplace, ecosystem, SDK

3.5 Solution Lifecycle States (Universal)

Every solution type in Qala moves through a governed lifecycle. The universal lifecycle states are defined below. Domain Packs may extend this set with domain-specific states, but all states must map to a universal state for cross-domain reporting.

State	Description	Entry Condition	Exit Conditions	Colour Code
Draft	Solution created; not yet ready for review	Solution record created	Configuration complete; owner submits for review	Grey
In Review	Formal review by designated reviewers is in progress	CCR submitted or manual transition	All reviewers approve or reject	Amber
Approved	Review complete; solution approved for release	All approvers sign off	Release created or state regressed	Blue
Active	Solution is in live production use and maintained	Release successfully deployed	Deprecation initiated or incident triggers review	Green
Deprecated	Solution approaching end of life; migration window open	EOL notice issued with migration plan	Archive date reached or migration complete	Orange
Recalled	Released version recalled due to critical defect or compliance failure	Recall event raised by Release Manager	Remediated version released or solution retired	Red
Archived	Solution or version immutably stored; no active work	Manual or policy-triggered archival	Terminal (or restored via archive restore process)	Dark Grey
Retired	Solution permanently discontinued; no restoration possible	Retirement approved by governance board	Terminal state; no further transitions	Black

3.6 Data Types Supported

The Qala data model supports the following primitive and composite types for all solution fields and message payloads:

Category	Types	Notes
Scalar	bool, string, char, varchar	All scalar types are nullable by default; explicitly mark non-nullable with constraint
Numeric	int, float, double	int supports 8/16/32/64-bit variants; float and double follow IEEE 754
Temporal	date, datetime, timestamp, duration	All temporal types stored in UTC; timezone metadata stored separately
Composite	array, tuple, set, map, object	Nested composites supported up to depth 10; deeper nesting requires explicit design review
Reference	pointer (UUID reference to another entity)	Pointers are validated at write time; dangling references are rejected
Custom	custom (user-defined via Domain Pack)	Domain Packs may define custom types with validation rules expressed in the Domain Pack YAML DSL
Null	null (explicit absence of value)	Null is distinct from empty string or zero; null means the value has not been set

4 Solution Factory

A Solution Factory is the primary organisational unit in Qala. It is a governed namespace that produces and manages Solution Development Environments, child factories, and all solution activity within a defined scope. Every user, team, or enterprise begins with at least one Solution Factory.

4.1 Factory Types & Tiers

Factory Type	Intended Use	Key Characteristics	Scale
Personal Factory	Single user managing personal projects, hobbies, and experiments	Single owner, unlimited SDEs, private by default, simplified governance	Solo
Team Factory	Small group building products or running shared projects	Multi-member, role-based access, shared templates, team change control	2–50 users
Organization Factory	Business managing multiple products, services, or internal tools	Org-level RBAC, department sub-factories, compliance policies, audit trails	51–5,000 users
Enterprise Factory	Large enterprise with complex governance and multi-domain solution estate	Hierarchical sub-factories, custom domain packs, SSO, data residency, SLA	5,000+ users
Platform Factory	Factory that produces reusable factory templates and domain packs for others	Public or gated; versioned factory blueprints; marketplace-publishable	Varies
Root Factory (Qala)	The Qala platform itself — the root of all factory hierarchies	System-level; produces all factory types; all domain packs; self-governing	Global

4.2 Factory Identity & Properties

Each Solution Factory carries a defined set of properties that govern its identity, ownership, and operational characteristics:

Property	Type	Description	Required
factory_id	UUID	Globally unique identifier, immutable after creation	Yes
name	string(255)	Human-readable factory name, unique within parent scope	Yes
slug	string(100)	URL-safe identifier, unique within tenant, immutable after activation	Yes
type	enum	personal | team | organization | enterprise | platform	Yes
tier	enum	1 (personal) | 2 (team) | 3 (org) | 4 (enterprise) | 5 (platform)	Yes
parent_id	UUID | null	Parent factory reference; null = root-level factory in tenant	No
owner_id	UUID	User or service account that owns this factory	Yes
description	text	Plain-text description of the factory's purpose and scope	No
policy	JSONB	Full governance policy: change control, release, versioning, audit	Yes
domain_pack_id	UUID | null	Active domain pack for this factory; null = core platform only	No

Property	Type	Description	Required
created_at	timestamptz	Factory creation timestamp (UTC, immutable)	Auto
updated_at	timestamptz	Last modification timestamp (UTC)	Auto

4.3 Factory Governance Policy

Each factory defines its own governance policy. This policy is inherited (and optionally overridden) by all child factories and SDEs within the factory's scope.

Policy Component	Description	Default
Change Control Policy	Defines which changes require CCRs, approval routing rules, and approval thresholds by risk level	All config changes require CCR; risk-based routing
Release Policy	Specifies which quality gates must pass before a solution can be released; automated vs. manual gates	All unit + integration tests passing; SAST clean
Version Strategy	SemVer (semantic versioning), CalVer (calendar versioning), or custom scheme	SemVer: Major.Minor.Patch
Audit Policy	Events to log, retention period, export schedule, and SIEM integration	All events; 7 years retention; monthly export
Access Control	RBAC roles defined at factory level; inherited by child factories and SDEs	Owner, Admin, Developer, Reviewer, Viewer
Compliance Frameworks	Regulatory frameworks active for this factory (e.g. ISO 27001, GxP, SOC 2)	Platform baseline (ISO 27001)
Notification Policy	Event types that trigger notifications; notification channels; escalation rules	Critical events; email + in-app; 24h escalation
Resource Quotas	Maximum SDEs, solutions, artefact storage, and compute per factory	Per-tier defaults; configurable with Enterprise plan

4.4 Factory as a Product

A configured factory — with its templates, domain packs, toolchains, and policies — can be packaged as a Factory Blueprint and shared through the Qala Marketplace. Organisations can publish blueprints for their industry (e.g. 'GxP-Compliant Pharma Factory Blueprint') which other organisations can instantiate and customise.

FACTORY BLUEPRINT	A Factory Blueprint contains: the factory type and tier configuration; all SDE templates pre-configured for the domain; domain pack reference (version-pinned); default toolchain definitions; governance policy presets; and an onboarding guide. Blueprints are versioned, signed by their publisher, and verifiable by consumers before instantiation.

4.5 RBAC Roles at the Factory Level

Role	Permissions	Scope
Factory Owner	Full control: create/delete SDEs, manage members, update policy, archive factory	All resources in factory and child factories
Factory Admin	Manage SDEs, manage members, update policy (except ownership transfer)	All resources in factory and child factories

Role	Permissions	Scope
Solution Manager	Create/update/delete solutions; manage CCRs; approve releases within SDE scope	Solutions and CCRs within assigned SDEs
Developer	Create/update solutions and artefacts; submit CCRs; run builds and tests	Solutions, artefacts, builds within assigned SDEs
Reviewer / Approver	Review and approve/reject CCRs and releases; view all solution content	CCRs and releases within assigned factories/SDEs
Viewer	Read-only access to all solution content, artefacts, and audit events	All content within factory scope
Auditor	Read-only access to all audit events, compliance reports, and evidence packages	Audit vault for assigned factory and below
Automation / Service Account	Programmatic API access; scoped to specific operations by service account policy	API-scoped; no UI access

5 Solution Development Environment (SDE)

THE SDE ANALOGY	The SDE is to Qala what a kernel is to an operating system: it provides the managed substrate on which solutions live, without being the solution itself. It is fully self-contained, versioned, and governable — a living workspace with its own lifecycle.

5.1 SDE Core Properties

Property	Description	Examples
Deployable	Can be instantiated on a platform, cloud environment, logical target designation, or physical machine	SaaS, AWS, Azure, GCP, on-premises, edge, air-gapped
Configurable	All aspects of the SDE are parameterised and adjustable without structural change	Environment variables, language runtimes, tool versions, profiles, feature flags
Distributable	Can be packaged, replicated, and instantiated in new contexts consistently	Consistent environments across teams, regions, and instances
Hermetic	Build environments within an SDE are fully isolated; all dependencies are frozen and immutable	No ambient credentials; digest-pinned dependencies; no 'latest' references
Versioned	Full version history of SDE configuration; any historical state can be inspected or restored	SDE manifest (.qala.yaml) is version-controlled; every config change is a new version

5.2 SDE Composition

Every SDE in Qala is composed of the following subsystems. All subsystems are defined in the SDE Manifest (.qala.yaml) and are version-controlled.

Subsystem	Description	Storage
Environment Variables	Runtime configuration values injected at SDE activation	SDE manifest; encrypted secrets in Vault
Languages & Runtimes	Programming languages, compilers, interpreters, and SDKs — all pinned to exact versions with digests	SDE manifest; cached in hermetic build registry
IDE & Editor Configs	Integrated development environment configurations, plugins, and workspace settings	SDE manifest; user-preference overlay
Toolchain	Hierarchical tooling organisation: Toolkits → Toolsets → Tools (see Chapter 5.5)	Toolchain registry; SDE manifest reference
Configuration Files	System-level and service-level configuration files — all version-controlled	SDE config repository
Profiles	Named configurations for different deployment targets or operational roles	SDE manifest; profile registry
Parameters & Options	Tuneable values and feature flags that modify SDE behaviour without structural change	SDE manifest; override mechanism for per-developer preferences
Content Management (CMS)	Solution-scoped versioned CMS for all content artefacts — playbooks, solution books, docs	CMS database; object storage for media
Solution Documentation	Charters, design docs, ADRs, and reference files specific to this SDE's solutions	CMS database; indexed by AI search

Subsystem	Description	Storage
Backup & Recovery	Scheduled snapshots, incremental backups, and restore points for all SDE state	Object storage; backup metadata in PostgreSQL

5.3 SDE Lifecycle States

State	Description	Entry Condition	Exit Condition
Provisioning	SDE being initialised; infrastructure being allocated and configured	SDE creation command accepted	Infrastructure ready; all checks pass → Active
Active	SDE is running and available for all solution work	Provisioning complete or reactivation	Manual suspend / policy trigger / quarantine event
Snapshotted	Point-in-time capture taken; SDE continues running	Snapshot command issued (manual or scheduled)	Snapshot complete; SDE returns to Active
Suspended	SDE paused; resources retained but not executing; reactivatable	Manual or policy trigger (e.g. inactivity timeout)	Manual reactivation command; decommission command
Rolled Back	SDE restored to a prior snapshot state; active from that snapshot	Rollback command issued against a specific snapshot	Rollback completes → Active from prior snapshot
Quarantined	SDE isolated by SEM due to confirmed security threat; no ingress/egress	SEM threat detection with confirmed severity	Security team clears threat; manual release from quarantine
Archived	SDE frozen and stored for audit purposes; no active work permitted	Manual archive or lifecycle policy trigger	Decommissioned (terminal) or restored via archive restore
Decommissioned	SDE permanently shut down; all resources released	Manual decommission or policy trigger post-archival	Terminal — no further transitions

5.4 SDE Governance Controls

Every SDE is subject to the following continuous governance controls, regardless of its type or domain:

- Ownership & RBAC: Each SDE has a designated owner. Ownership transfers are recorded in the immutable audit trail. All access changes are audit-logged with actor, timestamp, and reason.
- Change Control: All configuration changes to an SDE are tracked in a change log capturing: actor, timestamp, change type, and before/after diff. High-impact changes require a CCR.
- Policy Enforcement: Platform-wide security and compliance policies are continuously evaluated against all Active SDEs. Policy violations generate an alert and remediation checklist within 15 minutes.
- Lifecycle Approvals: Transitions to Archived or Decommissioned states require approval from the designated owner or a Factory Admin.
- Audit Trail: All lifecycle events — provisioning, snapshots, rollbacks, quarantine events, and decommissions — are immutably recorded in the Audit Vault.
- Drift Detection: SDE configuration is continuously compared against its version-controlled definition. Any detected drift generates an alert and requires resolution within the configured SLA.

5.5 SDE Manifest (.qala.yaml) — Specification

The SDE Manifest is the authoritative definition of an SDE's configuration. It is version-controlled, and every change to the manifest creates a new immutable version in the SDE's change history.

MANIFEST FORMAT

```
# Example .qala.yamlsde:  name: financial-core-dev  version: 2.3.1  domain: financial-services  domain_pack: qala/financial-services-v3runtime:  language: go  version: 1.22.0  digest: sha256:abc123...toolchain:  reference: enterprise-fintech-v2  version: 2.1.0policies:  inherit_from: enterprise-financial-factory  overrides:    ccr_required: true  auto_snapshot: dailyresources:  compute: standard-4  storage_gb: 100
```

5.6 SDE Types by Domain

SDE Type	Primary Technology	Key Requirements	Target Domains
Software Dev SDE	OCI container + Kubernetes orchestration	Hermetic builds, full IDE integration, fast provisioning (< 10 min), SAST/SCA scanning	Software, APIs, Platform, Tooling
Lab / Formulation SDE	VM or Kata container + NFS mounts	Commercial simulation software access, file-based toolchains, regulatory tool integration	Pharmaceutical, CPG, Materials Science
Financial Research SDE	OCI container + optional GPU attachment	Large dataset access (1TB+ storage), quantitative libraries (Python, R, Julia), backtesting framework	Financial Services, Investment, Risk
Edge / IoT SDE	Nix flake + bare-metal / microVM	Offline operation capability, cross-compilation, constrained resource targets, air-gap dependency cache	AgriTech, IoT, Industrial, Defence
Knowledge / Service SDE	Lightweight OCI container + CMS integration	Document versioning, AI retrieval, knowledge graph consistency, collaboration tooling	Professional Services, Consulting, Legal
Research / Academic SDE	OCI + large storage mounts + dataset registry	Dataset versioning, experiment reproducibility, HPC compute scaling, peer review workflow	Research, Academic, Pharma R&D
Business / Process SDE	Lightweight OCI + process modelling tools	BPMN/flowchart tooling, document management, process versioning, stakeholder collaboration	Business Ops, Process, Consulting

6 Solution Lifecycle Management

The Solution Lifecycle is the governed sequence of states and transitions that every solution in Qala passes through — from initial creation to final retirement. The lifecycle is enforced by the platform; invalid state transitions are rejected with descriptive errors.

6.1 The Nine-Stage Universal Solution Lifecycle

Stage	Description	Universal Activity	Governing Mechanism
1. Initiate	A new solution or version is proposed and registered in the platform	Solution record created; SDE assigned; owner set; initial CCR raised if required	FactoryAggregate policy validation
2. Configure	The solution and its environment are defined and versioned in the manifest	Configuration versioned; environment definition locked; model fields populated	SDE config version control; mandatory field validation
3. Develop	The solution is actively built, formulated, designed, or structured	All changes governed by CCR; artefacts versioned in registry; builds attested	CCRAggregate; build attestation engine
4. Test	The solution is validated against its specification and quality targets	Test plans executed; quality gates evaluated; defects tracked and resolved	Test Management Service; quality gate engine
5. Review	Formal review and approval prior to release	Approvers review; release record assembled; gate checklist completed	CCRAggregate approval chain; ReleaseAggregate
6. Release	The solution version is released and made available to consumers	Release event published; distribution channels notified; artefact signed and published	ReleaseAggregate; artefact signing pipeline
7. Distribute	The released solution is delivered to consumers or deployment targets	Distribution records created; consumer notification sent; deployment health monitored	Distribution Engine; deployment monitors
8. Govern	The live solution is monitored, audited, drift-detected, and maintained	Continuous drift detection; compliance evidence generated; AI recommendations issued	SEM; AI Agent; Audit Vault
9. Retire	The solution version or the solution itself is decommissioned	Deprecated state set; consumer migration window opened; archive event fired	Lifecycle policy engine; archive pipeline

6.2 State Machine — Valid Transitions

The following table defines all valid lifecycle state transitions. Any attempted transition not listed here is rejected by the platform with a 409 Conflict response and an audit event.

From State	Permitted Transitions	Blocking Conditions	Required Actor
Draft	In Review, Archived	Model required fields incomplete; SDE not active	Solution owner or Developer
In Review	Approved, Draft (regression), Archived	Pending required approvals; open blocking CCRs	Designated Reviewer / Approver
Approved	Active (via Release), In Review (regression), Archived	Release artefacts not ready; quality gates not passed	Release Manager

From State	Permitted Transitions	Blocking Conditions	Required Actor
Active	Deprecated, Recalled, In Review (via new CCR)	Open Critical defects; unresolved security alerts	Solution owner or Release Manager
Deprecated	Archived, Active (reversal of EOL)	Active consumer dependencies without migration plan	Factory Admin or Solution owner
Recalled	In Review (re-review after fix), Archived	Recall reason unresolved; replacement not ready	Release Manager + Factory Admin
Archived	Decommissioned	Active references from other solutions (must be resolved first)	Factory Admin
Decommissioned	(Terminal — no further transitions)	N/A	N/A

6.3 Version Management

Every solution in Qala follows a strict versioning scheme. The platform enforces version monotonicity — versions can only increase, never decrease. Rollback means activating a prior version, not changing version numbers.

Versioning Scheme	Format	Use Case	Increment Rules
Semantic Versioning (default)	Major.Minor.Patch (e.g. 3.2.1)	Software, APIs, Platforms, Tools, Frameworks	Major: breaking change; Minor: backward-compatible feature; Patch: backward-compatible fix
Calendar Versioning	YYYY.MM.DD or YYYY.MM.N (e.g. 2024.03.1)	Tax rule sets, regulatory updates, time-bound solutions	Date-based increment; sequential patch within a date
Revision Versioning	vN or N (e.g. v47)	Physical goods batch records, research versions, documents	Simple sequential increment; no semantic meaning
Build Versioning	N.N.N+buildmetadata (e.g. 3.2.1+20240309)	CI/CD artefacts with build metadata	SemVer base + build metadata appended (ignored in comparisons)

All versioning decisions are recorded in the Solution Factory's version strategy policy. The policy is enforced at release time — releasing a solution with a version that violates the configured strategy is rejected by the Release Gate.

6.4 Solution Playbook

Every solution has an associated Playbook — the design and decision record for that solution. The Playbook is automatically initialised when a solution is created and is maintained throughout the solution's lifecycle.

Playbook Section	Contents	Maintainer
Solution Charter	Purpose, scope, success criteria, stakeholders, and constraints for this solution	Solution Owner
Architecture Decision Records (ADRs)	All architectural decisions with context, options considered, decision made, and consequences	Tech Lead / Solution Architect
Design Documents	Technical design documents, API contracts, data model diagrams, and sequence diagrams	Development Team
Risk Register	Identified risks, likelihood, impact, mitigation strategies, and current status	Solution Owner + Risk Officer
Stakeholder Map	All stakeholders, their interests, communication preferences, and sign-off requirements	Solution Owner

Playbook Section	Contents	Maintainer
Change History	All CCRs raised against this solution — approved, rejected, and pending	Auto-populated by platform
Quality Benchmarks	Quality targets for test coverage, performance, security, and accessibility	Tech Lead + QA Lead
Lessons Learned	Post-release retrospective notes and improvement recommendations	Solution Team

6.5 Solution Book

The Solution Book is the comprehensive documentation repository for a solution. Unlike the Playbook (which focuses on design decisions), the Solution Book contains all operational and reference documentation.

- Auto-generated sections: API reference, configuration reference, changelog, dependency list, artefact manifest
- Team-authored sections: Getting started guide, architecture overview, runbooks, troubleshooting guides
- Compliance sections: Regulatory evidence packages, audit trail summaries, compliance certifications
- Training materials: Onboarding guides, tutorials, video scripts, FAQs

The Solution Book is versioned alongside the solution. Each solution release creates a new version of the Solution Book, which is searchable by the AI Agent and accessible via the CMS.

7 Change Control System

The Change Control Request (CCR) system is the governance backbone of Qala. Every significant change to a solution, SDE configuration, or platform entity is governed through a CCR — ensuring a consistent, auditable, risk-proportionate approval process.

7.1 CCR Types

CCR Type	Description	Default Risk	Required For
Standard	Pre-approved, low-risk changes with established procedures. Fast-tracked.	Low	Routine configuration updates; minor dependency bumps; documentation updates
Normal	Changes that require review and approval but are not time-critical	Medium	Feature additions; API contract changes; toolchain updates; security policy changes
Emergency	Time-critical changes required to restore service or prevent imminent harm	High	Security incident response; production outage remediation; critical compliance breach
Major	Changes with broad impact requiring Change Control Board review	High / Critical	Breaking API changes; solution type changes; factory policy changes; architectural decisions
Regulatory	Changes to regulated solutions requiring domain-specific approval chains	Domain-specific	GxP formulation changes; MiFID II model changes; FDA submission changes; tax rule activation

7.2 CCR Data Model

Field	Type	Description	Validation
ccr_id	UUID	Globally unique identifier, generated at submission	Auto-generated
title	string(500)	Human-readable description of the change	Required; min 10 chars
ccr_type	enum	Standard | Normal | Emergency | Major | Regulatory	Required
requester_id	UUID	User or service account submitting the CCR	Must have Developer role or above
solution_ids	UUID[]	All solutions directly affected by this change	At least one required; all must be in accessible SDEs
risk_score	decimal(0-100)	Platform-computed risk score based on change type, affected solutions, and history	Auto-computed; manual override requires Manager role
justification	text	Business justification for making this change	Required; min 50 chars
implementation_plan	text	Step-by-step plan for implementing the change	Required for Normal, Major, Regulatory
rollback_plan	text	Plan for reverting this change if it fails or causes regression	Required for Normal, Major, Regulatory
regulatory_implications	text | null	Any regulatory obligations created or affected by this change	Required for Regulatory type; optional for others

Field	Type	Description	Validation
evidence_ids	UUID[]	Supporting evidence documents attached to this CCR	Optional; required for Regulatory type
approval_chain	JSONB	Computed approval routing — list of required approvers in sequence	Auto-computed from factory policy and risk score
state	enum	Draft | Submitted | Under Review | Approved | Rejected | Deferred | Withdrawn | Implemented	Platform-managed state machine
created_at	timestamptz	CCR submission timestamp (UTC)	Auto-set

7.3 CCR Lifecycle State Machine

The CCR follows its own governed lifecycle. The state transitions are defined in the CCR state machine below.

From State	To State	Trigger	Actor
Draft	Submitted	Requester submits for review	Requester
Submitted	Under Review	First approver opens the CCR	First approver in chain
Under Review	Approved	All required approvers in chain approve	Final approver in chain
Under Review	Rejected	Any required approver rejects with documented reason	Any approver in chain
Under Review	Deferred	CCR deferred to next change window (planned maintenance)	Change Manager
Approved	Implemented	Change implemented and post-implementation review complete	Requester + Change Manager
Approved	Withdrawn	Requester withdraws before implementation (e.g. approach changed)	Requester
Rejected	Draft	Requester revises and resubmits (new CCR version)	Requester

7.4 Risk Scoring Algorithm

Every CCR is automatically assigned a risk score (0–100) based on the following weighted factors:

Factor	Weight	High-Risk Conditions	Low-Risk Conditions
Change Type	25%	Emergency or Regulatory type (max weight)	Standard type (min weight)
Affected Solution Count	20%	> 10 solutions affected; multi-factory impact	Single solution; isolated change
Solution Risk Profile	20%	Active production solutions; solutions in regulated domains	Draft or deprecated solutions; non-regulated domains
Historical Failure Rate	15%	> 20% failure rate for this change type on this solution	< 2% failure rate; well-established change pattern
Domain Risk Multiplier	10%	Pharmaceutical, financial, critical infrastructure	Personal projects, non-regulated software
Downstream Dependencies	10%	> 5 dependent solutions; cross-factory dependencies	No downstream dependencies

7.5 Multi-Stage Approval Chains

For domain-specific CCR workflows, Qala supports configurable multi-stage sequential approval chains. Examples:

Domain	Example Approval Chain	Emergency Override
Pharmaceutical (GxP)	Scientific Committee → Regulatory Affairs → Legal → Executive Board	CMO approval; mandatory post-implementation review within 24h
Financial Services (MiFID II)	Quantitative Analyst → Risk Officer → Compliance → Management Committee	Risk Officer + CRO approval; same-day post-implementation review
Tax Solutions	Tax Technologist → Legal → Jurisdiction Specialist → Effective Date Scheduler	Head of Tax approval; jurisdiction sign-off required within 48h
Agricultural / Regulatory	Agronomist → Environmental Officer → Regulatory Affairs → Commercial Lead	COO approval; environmental monitoring report within 72h
Software (Standard)	Tech Lead → Solution Manager → (Release Manager for Major)	On-call Manager; automated rollback pre-armed

8 Technology Stack & Service Topology

This chapter describes the complete technology stack and service decomposition for the Qala platform. Every technology choice is motivated by the binding architectural principles defined in Chapter 2.

8.1 Technology Stack by Layer

Layer	Technology	Version	Rationale
Backend Services	Go (primary)	1.22	Performance, low memory footprint, excellent concurrency primitives, fast compilation, single-binary deployment
AI / ML Workloads	Python	3.12	Dominant ML ecosystem; seamless integration with PyTorch, scikit-learn, HuggingFace, LangChain
Kernel Plane	Rust	1.78	Memory safety without GC; critical for security-sensitive code; zero-cost abstractions
Streaming Pipelines	Scala (Akka Streams)	3.4 / 2.9	Reactive streaming for high-throughput event pipelines; Akka persistence for event sourcing
API Gateway	Kong Gateway + custom Go middleware	3.7	Production-proven; plugin ecosystem; rate limiting; JWT validation; request routing
Message Broker	Apache Kafka (KRaft mode)	3.7	Persistent, ordered event log; at-least-once delivery; compaction for projection rebuilds; 100K+ events/sec throughput
Primary Database	PostgreSQL 16 + Citus	16 / 12.1	ACID compliance; row-level security for tenant isolation; JSONB for schema-flexible fields; horizontal sharding via Citus
Object Storage	S3-compatible (AWS S3 / Cloudflare R2 / MinIO)	—	Artefact storage; immutable versioned buckets; pre-signed URL access; 11-nine durability
Search Index	OpenSearch	2.x	Full-text and faceted search over solutions, playbooks, and solution books; AI vector search extension
Cache / Session	Redis (cluster mode)	7	Read projection cache; distributed rate-limit counters; session store; real-time pub/sub
Secrets Management	HashiCorp Vault / AWS Secrets Manager	1.16 / —	Dynamic secrets; per-tenant encryption keys; automated rotation; PKI for mTLS certificates
Service Mesh	Istio	1.21	mTLS enforcement; zero-trust service-to-service auth; circuit breaking; traffic shaping; observability injection
Container Runtime	Kubernetes	1.30	SDE runtime; factory namespace isolation via k8s namespaces + network policies; autoscaling via HPA
Observability	OpenTelemetry + Prometheus + Grafana + Loki	—	Unified trace/metric/log pipeline; vendor-neutral; exportable to any backend; dashboards as code
Frontend	React 19 + TypeScript + TanStack Query + Zustand	19 / 5 / —	Component-based; type-safe; optimistic UI updates; real-time via WebSocket subscription
Mobile	React Native (Expo SDK 52)	0.74	Cross-platform (iOS + Android); shared business logic with web; offline-capable with sync
CLI	Go (Cobra framework)	—	Single binary; cross-platform (Linux/macOS/Windows); auto-update; shell completion; grpc transport
IaC / Provisioning	Terraform + Helm	1.9 / 3.15	Declarative infrastructure; Helm for Kubernetes application packaging; Terragrunt for DRY module management

8.2 Domain Services Decomposition

Qala is implemented as a set of independently deployable domain services. Each service owns a clearly bounded domain, communicates via events on Kafka, and exposes a REST + GraphQL API through the API Gateway.

Service	Domain	Key Responsibilities	Persistent Store
Factory Service	Solution Factory	Factory CRUD, member management, policy enforcement, child factory hierarchy	PostgreSQL (factories table + events)
SDE Service	Development Environments	SDE provisioning, lifecycle management, configuration versioning, snapshot/rollback	PostgreSQL (sdes table + events) + k8s API
Solution Service	Solutions	Solution CRUD, lifecycle state machine, version management, model validation	PostgreSQL (solutions table + events)
CCR Service	Change Control	CCR submission, risk scoring, approval chain routing, state management	PostgreSQL (ccrs table + events)
Release Service	Releases	Release creation, quality gate evaluation, artefact signing, publish/recall	PostgreSQL (releases table + events) + S3
Identity Service	Auth / RBAC	User management, service account management, JWT issuance, RBAC enforcement	PostgreSQL (identity tables) + Redis (sessions)
Audit Service	Compliance	Immutable audit event recording, compliance report generation, evidence export	Append-only PostgreSQL partition + Kafka fan-out
CMS Service	Content	Playbook management, Solution Book generation, document versioning, AI indexing	PostgreSQL + S3 (media) + OpenSearch (index)
Artefact Service	Artefact Registry	Artefact storage, digest verification, signing, versioned retention	PostgreSQL (artefact metadata) + S3 (blobs)
Notification Service	Notifications	Event-driven notification delivery: email, in-app, Slack, Teams, webhook	PostgreSQL (notification log) + Redis (queue)
AI Agent Service	Intelligence	Recommendation engine, risk prediction, semantic search, drift analysis	PostgreSQL (AI state) + vector DB (embeddings)
Search Service	Discovery	Full-text search across all entities, AI-semantic search, faceted filtering	OpenSearch + embedding cache in Redis
Integration Service	Integrations	Third-party connectors, webhook engine, event bridge to external systems	PostgreSQL (integration config + event log)
Build Service	CI/CD	Hermetic build orchestration, pipeline execution, attestation generation	PostgreSQL (build records) + S3 (build logs)
Test Service	Quality	Test case management, test run execution, defect creation, quality gate eval	PostgreSQL (test data)

8.3 API Architecture

The Qala platform exposes three complementary API surfaces through a single API Gateway:

API Surface	Protocol	Use Case	Authentication
REST API v1	HTTPS / JSON	Primary API for all CRUD operations; supported by all clients and integrations	Bearer JWT (OAuth 2.0) or API Key
GraphQL API	HTTPS / JSON	Flexible queries for complex relationship traversal; used by the web app frontend	Bearer JWT (OAuth 2.0)

API Surface	Protocol	Use Case	Authentication
WebSocket API	WSS	Real-time event subscriptions; used by the web app for live updates (CCR state, build progress, alerts)	JWT via initial handshake; token refresh via connection
gRPC API	HTTPS/2 / Protobuf	High-performance service-to-service communication; used by CLI and SDK integrations	mTLS client certificates or service account JWT
Webhook Engine	HTTPS (outbound)	Outbound event delivery to external systems; any Qala event can trigger a webhook	HMAC-SHA256 signature on payload; target configures verification

8.4 Non-Functional Requirements Targets

Dimension	Target	Measurement Method	SLA Breach Action
API P99 Latency (read)	< 80ms under 1,000 RPS/tenant	OpenTelemetry trace percentiles; Grafana SLO dashboard	Auto-scale; incident created at 5-min sustained breach
API P99 Latency (write)	< 200ms for command acceptance	OpenTelemetry trace percentiles	Auto-scale; incident at 5-min sustained breach
Platform Availability	99.9% (Standard); 99.95% (Enterprise)	Uptime monitoring; monthly SLA report	Service credit issued; RCA within 24h
Event Processing Throughput	> 100,000 domain events/sec	Kafka consumer lag monitoring; throughput dashboards	Scale Kafka consumers; alert Engineering
Storage Durability	99.999999999% (11 nines)	S3/R2 SLA; multi-region replication verification	Disaster recovery runbook activated
Max SDE Boot Time	< 30 seconds from activation to first API response	Synthetic monitoring from activation event	Auto-retry; SDE health alert to Factory Admin
Tenant Onboarding	< 2 minutes from signup to first usable SDE	E2E synthetic test; onboarding funnel metrics	Incident raised; manual intervention within 5 min
RPO (Recovery Point)	< 1 minute (event log replication lag)	Continuous RPO measurement; DR drill validation	Disaster recovery runbook; status page update
RTO (Recovery Time)	< 15 minutes (full platform recovery)	DR drill results; quarterly validation	Multi-region failover; status page update
Security Scan SLA	Critical CVEs: 24h; High CVEs: 72h	Snyk / Trivy pipeline; SLA tracking dashboard	Automatic quarantine of affected service; emergency patch process

9 Data Architecture & Event Sourcing

Qala uses event sourcing as its primary persistence strategy. Every domain entity is represented as an ordered sequence of immutable events. Current state is a projection derived by replaying events. This chapter defines the canonical event schemas, aggregate boundaries, projection models, and storage layout.

9.1 Seven Root Aggregates

The following are the seven root aggregates in Qala. Each aggregate has exclusive ownership of its state and exposes a command interface. Cross-aggregate operations are coordinated via domain events and sagas — never by direct aggregate coupling.

Aggregate	Root Entity	Invariants Enforced	Commands Accepted
FactoryAggregate	SolutionFactory	Factory name unique per parent; max nesting depth ≤ 20; policy cannot be removed while active SDEs exist	CreateFactory, UpdatePolicy, ArchiveFactory, AddMember, RemoveMember, AttachDomainPack
SDEAggregate	SDE	SDE name unique per factory; version strictly monotonic; cannot activate with missing required toolchain	CreateSDE, ActivateSDE, SuspendSDE, ArchiveSDE, UpdateConfig, AttachToolchain, ScaleSDE
SolutionAggregate	Solution	Version strictly monotonic; cannot transition backwards (except rollback); open CCR blocks direct state change	CreateSolution, TransitionState, RollbackToVersion, AttachPlaybook, AttachSolutionBook
CCRAggregate	ChangeControlRequest	Cannot be approved if required evidence missing; cannot re-open after Implemented; impact scope non-empty	SubmitCCR, ApproveCCR, RejectCCR, DeferCCR, WithdrawCCR, AttachEvidence, ImplementCCR
ReleaseAggregate	Release	Release artefact SHA-256 must be verified; required approvals must be complete; all quality gates passed	CreateRelease, AddArtefact, ApproveRelease, PublishRelease, RecallRelease
IdentityAggregate	User / ServiceAccount	Username/email unique per tenant; service accounts cannot own factories; MFA state consistent with policy	CreateUser, AssignRole, RevokeRole, EnableMFA, SuspendUser, CreateServiceAccount
ModelAggregate	SolutionModel	Model name unique per domain pack version; lifecycle states must form a valid DAG; no circular transitions	CreateModel, PublishModel, DeprecateModel, UpdateLifecycle, AttachDomainPack

9.2 Canonical Event Envelope Schema

Every domain event in Qala conforms to a canonical envelope schema. The payload is event-type specific. Events are serialised as JSON and stored in Kafka topics (partitioned by aggregate ID) and in the PostgreSQL event store.

EVENT SCHEMA

```
{ "event_id": "uuid-v7", // Time-ordered UUID (sortable) "event_type": "solution.state_transitioned", "event_version": "1", // Schema version for this event type "aggregate_id": "uuid", // Root aggregate instance ID "aggregate_type": "SolutionAggregate", "tenant_id": "uuid", // Tenant scope (for isolation) "factory_id": "uuid", // Factory scope "correlation_id": "uuid", // Ties related events (e.g. CCR + Solution) "causation_id": "uuid", // ID of command that caused this event "actor_id": "uuid",
```

```
// User or service account that triggered  "timestamp":       "2025-09-14T12:00:00.000Z",  "schema_version":
"qala/v1",  "payload":         { ... }              // Event-type specific data}
```

9.3 Core Event Catalogue

Event Type	Aggregate	Payload Fields	Downstream Effects
factory.created	FactoryAggregate	factory_id, name, type, parent_id, owner_id, policy	Create factory projection; provision tenant namespace; send welcome notification
factory.policy_updated	FactoryAggregate	factory_id, old_policy, new_policy, reason	Update factory projection; evaluate active SDEs against new policy
sde.created	SDEAggregate	sde_id, factory_id, name, domain, config, manifest	Create SDE projection; provision k8s namespace; initialise repositories
sde.activated	SDEAggregate	sde_id, activated_at, toolchain_snapshot	Deploy SDE workloads; register health check; send activation notification
sde.config_updated	SDEAggregate	sde_id, changed_keys, old_config, new_config	Update SDE projection; trigger config reload in SDE runtime; audit event
solution.created	SolutionAggregate	solution_id, model_id, sde_id, fields, owner_id	Create solution projection; initialise playbook stub; initialise solution book
solution.state_transitioned	SolutionAggregate	solution_id, from_state, to_state, actor_id, reason	Update solution projection; check quality gates; emit notifications
solution.version_published	SolutionAggregate	solution_id, old_version, new_version, changelog	Create immutable version snapshot; index new version; update version history
solution.rolled_back	SolutionAggregate	solution_id, target_version, rollback_reason	Update projection to target version; open post-rollback CCR
ccr.submitted	CCRAggregate	ccr_id, solution_id, type, risk_score, description	Create CCR projection; compute approval routing; send approver notifications; block solution transitions
ccr.approved	CCRAggregate	ccr_id, approver_id, stage, conditions, timestamp	Update CCR projection; if final stage: unblock solution; create ccr.fully_approved event
release.created	ReleaseAggregate	release_id, solution_id, version, artefact_ids, target	Create release projection; begin quality gate evaluation pipeline
release.published	ReleaseAggregate	release_id, published_at, distribution_targets, attestation	Mark release immutable; push artefacts to targets; update solution state to Released
release.recalled	ReleaseAggregate	release_id, reason, actor_id, recall_timestamp	Create recall event; notify all deployment targets; update solution to Recalled state

9.4 Projection Models

Read projections are maintained as materialised views in PostgreSQL and cached in Redis. Each projection is rebuilt by replaying events from the Kafka topic. Target consistency lag under nominal load: < 500ms.

Projection	Primary Table	Key Columns	Rebuild Trigger
SolutionView	solutions_view	id, tenant_id, factory_id, sde_id, model_id, name, state, version, owner_id, domain, tags, updated_at	solution.* events
FactoryView	factories_view	id, tenant_id, parent_id, name, type, member_count, sde_count, policy_hash, created_at	factory.* events

Projection	Primary Table	Key Columns	Rebuild Trigger
SDEView	sdes_view	id, tenant_id, factory_id, name, state, version, domain, toolchain_ids, scale_config, manifest_hash	sde.* events
CCRView	ccrs_view	id, tenant_id, solution_id, type, state, risk_score, submitter_id, current_stage, approver_ids	ccr.* events
ReleaseView	releases_view	id, tenant_id, solution_id, version, state, artefact_count, published_at, recalled_at	release.* events
AuditView	audit_events_view	id, tenant_id, actor_id, event_type, aggregate_id, timestamp (append-only, no update)	All events (fan-out projection)
ActivityFeedView	activity_feed_view	tenant_id, factory_id, event_type, actor_id, summary_text, timestamp (rolling 90-day window)	Selected activity-relevant events

9.5 Kafka Topic Architecture

Topic	Domain	Retention	Partitioning Key	Key Event Types
sde_events	Platform	30 days	sde_id	SDE_CREATED, ACTIVATED, SNAPSHOTTED, QUARANTINED, ROLLED_BACK, ARCHIVED
solution_events	Universal	90 days	solution_id	SOLUTION_CREATED, VERSION_RELEASED, STATE_CHANGED, RECALLED, RETIRED
build_events	Execution	90 days	build_id	BUILD_STARTED, PASSED, FAILED, ATTESTATION_CREATED, DEPENDENCY_VIOLATION
release_events	Execution	90 days	release_id	RELEASE_PLANNED, GATE_PASSED, GATE_FAILED, DEPLOYED, ROLLED_BACK, RECALLED
ccr_events	Governance	365 days	ccr_id	CCR_SUBMITTED, APPROVED, REJECTED, DEFERRED, IMPLEMENTED
security_events	Security	365 days	sde_id or solution_id	THREAT_DETECTED, VULNERABILITY_FOUND, QUARANTINE_APPLIED, CLEARED
audit_events	Compliance	7 years	tenant_id	ALL events fan-out (append-only compliance record)
ai_events	Intelligence	30 days	entity_id	RECOMMENDATION_GENERATED, RISK_SCORE_UPDATED, DRIFT_PREDICTED

10 Security Architecture

Security in Qala is not a layer added on top — it is a foundational architectural property. The platform is designed around zero-trust principles, immutable audit trails, and continuous threat detection. This chapter defines the security architecture across identity, network, data, and runtime layers.

10.1 Security Model Overview

ZERO-TRUST PRINCIPLE	All service-to-service communication in Qala is mutually authenticated (mTLS). No implicit trust exists between services. Every request carries a verifiable identity. Network-level segmentation is a defence-in-depth measure, not a primary security control. The service mesh (Istio) enforces mTLS at the transport layer for all inter-service communication.

10.2 Identity & Authentication

Identity Type	Authentication Method	Token Format	Scope
Human Users (web/mobile)	OAuth 2.0 + OIDC with optional SSO (SAML 2.0)	JWT (RS256); 1-hour expiry; refresh token (7-day)	User-scoped; tenant-scoped claims in JWT
Human Users (CLI)	OAuth 2.0 device flow or API key	JWT (short-lived) or API key (long-lived, scoped)	User-scoped; API key scoped to specific operations
Service Accounts	Client credentials + mTLS certificate	JWT (short-lived, 15-minute) + mTLS certificate	Service-scoped; specific endpoint whitelist in policy
CI/CD Pipelines	OIDC workload identity (GitHub Actions, GitLab CI)	JWT issued by CI provider; verified against OIDC endpoint	Build-scoped; read/write artefacts only
External Integrations	API key or OAuth 2.0 client credentials	API key (256-bit entropy) with HMAC request signing	Integration-scoped; per-integration permission set
Platform Services (internal)	mTLS mutual certificates issued by Vault PKI	X.509 certificates; auto-rotated every 24h by Vault	Service-to-service; no external access

10.3 RBAC Model

Qala uses a hierarchical RBAC model. Permissions are defined at the platform level, roles are defined at the factory level, and role assignments are made at the factory, SDE, or solution level. Permissions cascade downward through the hierarchy unless explicitly restricted.

Permission Category	Permissions Included	Minimum Role
Solutions	solution:read, solution:create, solution:update, solution:delete, solution:transition, solution:release	Viewer (read only); Developer (create/update); Manager (transition/release)
CCRs	ccr:read, ccr:submit, ccr:approve, ccr:reject, ccr:implement	Viewer (read); Developer (submit); Approver (approve/reject); Manager (implement)
SDEs	sde:read, sde:create, sde:update, sde:archive, sde:decommission, sde:config_update	Viewer (read); Factory Admin (create/update/archive)
Factories	factory:read, factory:create, factory:update, factory:archive, factory:manage_members	Viewer (read); Factory Owner (all)

Permission Category	Permissions Included	Minimum Role
Artefacts	artefact:read, artefact:write, artefact:delete, artefact:sign, artefact:verify	Viewer (read/verify); Developer (write); Release Manager (sign/delete)
Audit Events	audit:read, audit:export, audit:verify_integrity	Auditor (all audit permissions); no write permissions exist for audit events
Admin	platform:manage_users, platform:manage_integrations, platform:manage_domain_packs	Platform Admin only

10.4 Data Security

Data Category	Encryption at Rest	Encryption in Transit	Access Control
Solution content and metadata	AES-256-GCM; per-tenant key in Vault	TLS 1.3 minimum	RLS in PostgreSQL; tenant_id claim in JWT
Artefacts (binaries, packages)	AES-256-GCM via S3 SSE-KMS; per-tenant key	TLS 1.3 + pre-signed URL with expiry	Pre-signed URL with time-limited access; artefact:read permission
Secrets and credentials	Vault Transit encryption; per-secret key rotation	mTLS to Vault; TLS 1.3 for external APIs	Vault policy + JWT claims; no direct DB access
Audit events	AES-256-GCM; audit key in dedicated Vault mount	TLS 1.3 + internal mTLS	audit:read permission; append-only by platform services
AI model data and embeddings	AES-256-GCM; per-tenant key	Internal mTLS	AI Agent service account; tenant-scoped query filters
Event log (Kafka)	AES-256-GCM; Kafka at-rest encryption; per-partition key rotation	mTLS for all Kafka producer/consumer connections	SASL + ACLs; per-service topic permissions

10.5 Security Event Monitor (SEM)

The Security Event Monitor is a dedicated platform service in the Intelligence Plane that provides continuous threat detection and automated response across all SDEs and solutions.

SEM Capability	Detection Method	Automated Response	Alert Level
Dependency Vulnerability Scanning	Continuous Snyk/Trivy SCA scanning against CVE databases	Block build pipeline; notify Developer + Security; create remediation CCR	Critical: block; High: alert; Medium: log
SAST Code Analysis	Static analysis on every code commit using Semgrep + CodeQL	Flag commit; prevent merge to main; create defect record	Critical: block commit; High: flag for review
Container Image Scanning	Trivy image scanning on every build	Block promotion beyond build stage; create remediation task	Critical: block; High: notify; Medium: log
Runtime Threat Detection	Falco rules on all SDE container runtimes; eBPF-based syscall monitoring	Quarantine SDE; isolate network; alert Security team	Critical: auto-quarantine; High: alert + monitor
Network Anomaly Detection	Istio telemetry + ML anomaly detection on traffic patterns	Rate limit suspicious traffic; create incident; alert Security	High: alert; Medium: log + monitor
Secrets Leak Detection	Git hooks + runtime scanning for committed secrets or credential exposure	Block commit; revoke exposed credential; notify owner	Critical: immediate revoke + notify

SEM Capability	Detection Method	Automated Response	Alert Level
Compliance Drift Detection	Continuous policy evaluation against all active SDEs	Alert Factory Admin; create remediation task with SLA	High: alert with 15-min SLA; Medium: 24-hour SLA

11 AI & Intelligence Layer

The Qala AI Agent is a domain-aware intelligence layer woven through every platform domain. It is advisory — never blocking primary workflows — and operates under strict privacy and transparency controls. The agent is built on the Intelligence Plane (Rust) with domain-specific models hot-swappable per solution type.

11.1 AI Agent Design Principles

AI AS CO MPA NIO N	The AI Agent surfaces recommendations inline — dismissable, actionable, contextual. It never blocks a primary workflow. In regulated domains (pharmaceutical, financial, legal), AI is strictly advisory; human approval is always the final gate. The AI appears when useful, disappears when not. All AI recommendations are labelled, attributed, and auditable.

11.2 AI Capabilities by Domain

Capability	Trigger	Output	Domain Availability
CCR Risk Prediction	CCR description entered before submission	Predicted risk score (0–100) with confidence; similar historical CCRs; suggested approvers	All domains
Change Impact Analysis	Solution version bump or CCR submission	Downstream dependency impact map; affected solutions list; suggested test scope	All domains
Test Case Generation	Code commit or solution version change	Suggested test cases for changed areas; coverage gap analysis	Software, Research, Financial
Configuration Drift Prediction	Historical drift analysis on active SDEs	Predicted drift risk score; suggested preventive configuration locks	All domains
Semantic Search	User query in search bar or Command Palette	Ranked results across all solution content using semantic similarity	All domains
Documentation Auto-generation	New solution version published	Draft Solution Book sections from artefact analysis; changelog from commit diffs	Software, Research
Formulation Optimisation	Ingredient specification change in pharma SDE	Suggested alternative excipient combinations; regulatory impact summary	Pharmaceutical (CPG Domain Pack)
Financial Model Validation	Model parameter change in financial SDE	Backtesting summary; risk metric comparison to previous version; regulatory alignment check	Financial (Financial Domain Pack)
Compliance Gap Analysis	New regulation ingested or compliance framework updated	Gap report: which solutions are potentially non-compliant; remediation priority list	All regulated domains
Onboarding Acceleration	New Developer added to a factory or SDE	Personalised getting-started guide; relevant playbook sections; similar solutions to reference	All domains

11.3 AI Agent Architecture

The AI Agent is deployed as a dedicated service in the Intelligence Plane. It consumes domain events from Kafka and generates recommendations that are stored in the AI state database and delivered to users through the API.

Component	Technology	Responsibility
Event Consumer	Kafka consumer (Rust)	Consumes domain events from all Kafka topics; routes to relevant inference pipelines
Inference Engine	PyTorch + HuggingFace (Python sidecar)	Runs domain-specific ML models for risk scoring, anomaly detection, and NLP tasks
Semantic Search Engine	OpenSearch + vector embeddings	Indexes all solution content; supports semantic similarity search via embedding vectors
Recommendation Store	PostgreSQL (ai_recommendations table)	Stores generated recommendations with confidence scores, expiry, and dismissal tracking
Domain Model Registry	Model registry (MLflow or equivalent)	Versioned, deployable domain-specific models; hot-swap capability without service restart
Privacy Filter	Rust middleware in Intelligence Plane	Strips PII and sensitive content before any model inference; GDPR compliance enforcement
Audit Logger	Async write to audit_events Kafka topic	Every AI recommendation generated, dismissed, or acted upon is recorded as an audit event

11.4 AI Transparency & Privacy Controls

Control	Description	Configuration Level
Recommendation Attribution	Every AI recommendation identifies the model version, confidence score, and data sources used to generate it	Platform-enforced; always on
Dismissal Tracking	Users can dismiss any AI recommendation; dismissals are recorded and feed back into model retraining	Platform-enforced; always on
AI Opt-out	Individual users or entire factories can opt out of AI recommendations. Opt-outs are respected within 5 minutes.	User preference (individual) or Factory Admin policy (factory-wide)
Data Residency for AI	AI inference for a tenant only uses data from that tenant's event stream; no cross-tenant data in inference	Platform-enforced; tenant-scoped encryption keys apply to AI processing
Human-in-the-Loop	Regulated domains (pharmaceutical, financial, tax) enforce human approval as the final gate; AI is advisory only	Domain Pack configuration; cannot be overridden by individual users
Model Versioning	All production AI models are versioned and recorded in the Domain Model Registry; rollback to prior version available	AI Platform Admin
Right to Explanation	Users can request an explanation of any AI recommendation; the platform returns a human-readable rationale	User-accessible via recommendation detail panel

12 Release & Distribution Management

Release management in Qala governs the process of packaging, validating, signing, and publishing a solution version from development through to distribution channels. Every release is a first-class, immutable entity with its own lifecycle and governance trail.

12.1 Release Lifecycle

Release State	Description	Entry Condition	Exit Conditions
Draft	Release record created; artefacts being assembled and attached	Release Manager creates release record for a solution version	All required artefacts attached; Release Manager submits for gate evaluation
Gate Evaluation	Quality gates being automatically evaluated	Release submitted; automated gate pipeline started	All gates pass (→ Approved) or any gate fails (→ Draft with failing gates flagged)
Approved	All quality gates passed; awaiting final deployment authorisation	All required quality gates pass; manual approval (if configured) signed off	Publish command issued or release cancelled
Published	Release deployed to all configured distribution targets; immutable	Publish command accepted; artefacts distributed to targets	Terminal (or Recalled if critical issue found post-deployment)
Recalled	Release pulled from all distribution targets due to critical defect or compliance issue	Recall command issued by Release Manager or automatic SEM trigger	Replacement release published or solution retired

12.2 Quality Gates

Quality gates are automated checks that must pass before a release can be approved. Gates are defined in the Solution Factory's Release Policy and can be customised per domain.

Gate	Type	Pass Condition	Domain Applicability
Unit Test Coverage	Automated	Coverage >= configured threshold (default: 80%)	Software, Research
Integration Tests	Automated	100% of integration test suite passing	Software, APIs, Platform
SAST Clean	Automated	No Critical or High severity SAST findings unresolved	Software, Firmware
Dependency Vulnerability	Automated	No Critical CVEs; High CVEs have documented risk acceptance	Software, Firmware
Container Image Scan	Automated	No Critical vulnerabilities in container base image	Software (containerised)
Performance Baseline	Automated	P99 latency within 10% of previous version; no throughput regression	Software, APIs, Platform
Artefact Integrity	Automated	All artefact SHA-256 digests verified; SLSA attestation present and valid	All solution types
CCR Completeness	Automated	All open CCRs for this version are in Implemented or Closed state	All solution types
Release Notes Present	Automated	Changelog and release notes attached to release record	All solution types

Gate	Type	Pass Condition	Domain Applicability
Regulatory Evidence	Manual / Automated	Required regulatory documents attached and approved (domain-specific)	Regulated domains only

12.3 Artefact Signing & Attestation

All release artefacts are cryptographically signed and accompanied by a SLSA-compatible build attestation record. Attestation is verified at every distribution gate and at deployment time.

Attestation Field	Description
build_id	Unique identifier for this specific build invocation
source_commit	Git commit SHA of the source code at build time (immutable reference)
env_definition_hash	SHA-256 digest of the build environment definition used (hermetic verification)
builder_identity	Identity of the build agent / CI runner that executed the build (OIDC-verified)
inputs_hash	Merkle hash of all declared inputs to the build (supply chain integrity)
outputs_hash	SHA-256 digest of each produced artefact (tamper detection)
timestamp	UTC timestamp of build completion (RFC3339)
signature	Cryptographic signature over the attestation record using the platform signing key
provenance_standard	Attestation format compatibility declaration (SLSA Provenance v1.0)
solution_id	Qala solution ID this artefact belongs to (platform linkage)
release_id	Qala release ID this artefact is part of (release linkage)

12.4 Distribution Channels

Channel Type	Mechanism	Authentication	Applicable Solution Types
Container Registry	OCI-compliant image push; SHA-256 digest pinning	Registry credentials rotated by Vault; image signed with Cosign	Software (containerised), Firmware
Package Registry	OCI artefact or native package format (Maven, npm, PyPI, etc.)	Signed package with publisher certificate	Libraries, SDKs, Packages, Frameworks
Deployment Target (k8s)	Helm chart deployment via GitOps pipeline (ArgoCD)	k8s RBAC; Helm release managed by ArgoCD service account	Software, APIs, Platform
Qala Marketplace	Solution Blueprint published to Marketplace registry	Publisher certificate + Marketplace approval workflow	All solution types (public sharing)
Direct Download (secure)	Pre-signed S3 URL with expiry; audit log on access	User authentication required; download logged to audit vault	Physical Goods specs, Research, Docs
Regulatory Submission	Structured data package assembled and submitted via domain-specific adapter	Electronic signature (21 CFR Part 11 compliant); submission audit trail	Pharmaceutical, Tax, Financial (regulated)

13 Deployment Architecture

Qala is architected for flexible deployment across multiple models — from a fully managed multi-tenant SaaS to an air-gapped on-premises installation. Every deployment model uses the same platform codebase; deployment configuration adjusts the operational characteristics.

13.1 Deployment Models

Model	Description	Infrastructure Ownership	Best For
Qala SaaS (Multi-tenant)	Managed by Qala. Shared infrastructure with per-tenant logical isolation via RLS and namespace separation	Qala manages all infrastructure	Individuals, SMBs, most enterprises
Qala SaaS (Single-tenant)	Managed by Qala. Dedicated infrastructure for one customer. Full physical isolation.	Qala manages dedicated infrastructure	Enterprises with strict data isolation requirements
Self-Hosted (Cloud)	Customer installs Qala on their own cloud account (AWS, Azure, GCP) using Qala-provided Helm chart and Terraform modules	Customer owns cloud account; Qala provides IaC	Enterprises requiring data sovereignty or custom cloud configuration
Self-Hosted (On-Premises)	Customer installs Qala on their own data centre infrastructure using bare-metal or private cloud Kubernetes	Customer owns and manages all infrastructure	Regulated industries with on-premises mandates (defence, government, healthcare)
Air-Gapped	Fully isolated installation with no external network access. Manual update packages. Offline dependency cache.	Customer manages completely isolated environment	Classified environments, critical infrastructure, maximum security requirements
Edge / Embedded	Lightweight Qala node deployed to edge devices (IoT gateways, field equipment, point-of-sale systems)	Customer owns edge devices; Qala provides edge runtime	Agricultural IoT, point-of-sale, remote lab or field operations

13.2 Kubernetes Architecture

Qala's platform services and SDE workloads run on Kubernetes. The following namespace structure provides isolation between platform services and tenant workloads:

Namespace	Contents	Network Policy
qala-platform	Core platform services: Factory, SDE, Solution, CCR, Release, Identity, Audit, CMS, AI Agent services	Allow intra-namespace; allow from qala-gateway; deny all external ingress
qala-gateway	API Gateway (Kong), rate limiter, SSL termination, CDN origin	Allow ingress from internet (port 443 only); allow egress to qala-platform
qala-data	PostgreSQL (Citus), Redis cluster, Kafka cluster, OpenSearch cluster	Allow from qala-platform and qala-intelligence only; no external access
qala-intelligence	AI Agent service, SEM service, vector database	Allow from qala-platform; no external ingress; allow egress to AI model endpoints (allow-listed)
tenant-{tenant_id}	One namespace per tenant's SDE workloads: each SDE gets a sub-namespace or pod label group	Allow intra-tenant; deny cross-tenant; allow egress to qala-platform APIs only

Namespace	Contents	Network Policy
qala-observability	Prometheus, Grafana, Loki, OpenTelemetry Collector, Jaeger	Allow scrape from all namespaces; UI accessible to platform admins only
qala-vault	HashiCorp Vault cluster for secrets management and PKI	Allow from qala-platform and qala-intelligence; deny all other namespaces

13.3 High Availability & Disaster Recovery

Component	HA Configuration	RPO	RTO
API Gateway (Kong)	Active-active, minimum 3 replicas across 3 AZs; health check failover < 10s	N/A (stateless)	< 10 seconds (automatic failover)
Domain Services (Go)	Active-active, minimum 2 replicas; HPA based on RPS; anti-affinity rules across AZs	N/A (stateless; event store is source of truth)	< 30 seconds (pod restart or reschedule)
PostgreSQL (Citus)	Primary + 2 read replicas per shard; synchronous replication to replica; Patroni for automatic failover	< 1 minute (sync replication lag)	< 2 minutes (Patroni automatic failover)
Kafka (KRaft)	3-node KRaft cluster; replication factor 3; min.insync.replicas = 2	< 1 minute (lag from last committed offset)	< 2 minutes (partition leader re-election)
Redis Cluster	6-node cluster (3 primary, 3 replica); automatic failover via Redis Sentinel	Up to last snapshot (AOF persistence)	< 30 seconds (Sentinel failover)
Object Storage (S3)	Multi-region replication; versioned buckets; immutable object lock for artefacts	Zero (synchronous cross-region replication)	Immediate (cross-region failover via DNS)
HashiCorp Vault	Active-active HA with 5-node Raft cluster; auto-unseal via cloud KMS	Near-zero (Raft log replication)	< 30 seconds (Raft leader election)
Full Platform Recovery	All services recoverable from event log replay and backup restoration	< 1 minute (event log RPO)	< 15 minutes (full platform RTO)

14 Integration Architecture

Qala integrates with the existing tool landscape through a standardised integration framework. Integrations are first-class citizens in the SDE toolchain system — versioned, governed, and auditable.

14.1 Integration Categories

Category	Description	Examples	Integration Method
Native Integrations	Maintained by Qala for most common platforms; full feature support; auto-updated	GitHub, GitLab, Jira, Confluence, Slack, Teams, SAP, Salesforce, ServiceNow, AWS, Azure, GCP	Native connector in Integration Service; factory-level configuration
Domain Integrations	Industry-specific integrations for regulated or specialised domains	LIMS (pharma), Bloomberg (financial), EPCIS (supply chain), FAOSTAT (agricultural), EDGAR (financial regulatory)	Domain Pack integration bridge; requires Domain Pack to be active
API-First	Any tool integrating via REST or GraphQL APIs; full platform capability access	Custom internal tools, vendor platforms, bespoke enterprise systems	OAuth 2.0 client credentials or API key; REST/GraphQL
Webhook Engine	Any Qala event can trigger an outbound webhook to any external system	External notification systems, event-driven automation, SIEM feeds	Webhook configuration in Integration Service; HMAC signing
Import / Export	Solutions, SDEs, and factories can be exported as structured packages and imported	Cross-platform migration, factory blueprint sharing, backup and restore	CLI or API; package format documented as open specification

14.2 Native Integration Catalogue

Integration	Category	Capabilities	Auth Method
GitHub	SCM / CI	Source code management, GitHub Actions pipeline trigger, PR-to-CCR linking, repository-level SDE binding	GitHub App (fine-grained permissions)
GitLab	SCM / CI	Source code management, GitLab CI pipeline, merge request-to-CCR linking	GitLab App OAuth + personal access token
Jira	Issue Tracking	Defect sync, CCR-to-Jira issue linking, sprint board integration, release ticket auto-creation	Jira OAuth 2.0 (3LO)
Confluence	Documentation	Solution Book page sync, Playbook page publishing, space integration	Confluence OAuth 2.0 (3LO)
Slack	Notifications	CCR notifications, build status, release alerts, SEM security alerts to dedicated channels	Slack app OAuth + Incoming Webhooks
Microsoft Teams	Notifications	CCR notifications, build status, compliance alerts to Teams channels	Teams bot registration + Incoming Webhooks
AWS	Cloud Infra	SDE provisioning on EKS, S3 artefact storage, Secrets Manager integration, CloudTrail audit feed	IAM role + OIDC workload identity

Integration	Category	Capabilities	Auth Method
Azure	Cloud Infra	SDE provisioning on AKS, Blob Storage artefact storage, Key Vault secrets, Azure Monitor	Azure managed identity + service principal
GCP	Cloud Infra	SDE provisioning on GKE, Cloud Storage, Secret Manager, Cloud Audit Logs	GCP Workload Identity Federation
SAP	ERP	Solution record sync for physical goods, production order linkage, material master integration	SAP OAuth 2.0 + BTP service binding
Salesforce	CRM	Solution-to-opportunity linkage, CPQ integration, account-level factory provisioning	Salesforce Connected App (OAuth 2.0)
ServiceNow	ITSM	CCR bidirectional sync, incident-to-security-event linking, change management integration	ServiceNow OAuth 2.0 + REST API

14.3 Webhook Architecture

The Webhook Engine allows any Qala domain event to trigger an outbound HTTP call to any registered external endpoint. Webhooks are a first-class feature — not a bolt-on — and are versioned, retried, and audited.

Feature	Description
Event Selection	Any event type from any Kafka topic can trigger a webhook. Filtering by event type, factory, SDE, solution, or tag.
Delivery Guarantees	At-least-once delivery with exponential backoff retry (1s, 2s, 4s, 8s, ... up to 1h); dead letter queue after 10 failed attempts.
Security	HMAC-SHA256 payload signature with Qala-provided secret; signature included in X-Qala-Signature header for verification by receiver.
Versioning	Webhook payload format is versioned; consumers can pin to a specific payload schema version.
Observability	Every webhook delivery attempt (success and failure) is recorded in the integration event log and accessible via API.
Testing	Test delivery endpoint available; sends a sample event payload to verify webhook configuration without triggering real events.

15 Domain Extensions (Domain Packs)

Domain Packs are the primary mechanism for extending Qala beyond the platform core to support industry-specific solution types, workflows, compliance frameworks, and integrations. Every Domain Pack is independently versioned, deployable, and community-contributable.

15.1 Domain Pack Architecture

Component	Description	Format
Solution Type Schemas	Extended solution type definitions with domain-specific attributes, validations, and relationships	YAML DSL with optional Go extension for complex validation
Lifecycle State Machine Overlay	Additional lifecycle states and transition rules specific to the domain (must map to universal states)	YAML state machine definition
CCR Workflow Template	Domain-specific approval chain template with named roles and routing rules	YAML workflow definition
Compliance Framework Adapter	Mapping between platform events and compliance evidence requirements for specific frameworks	Go plugin + YAML evidence map
Integration Bridge Config	Pre-configured connectors for domain-specific external systems	YAML connector configuration
Vocabulary Overlay	UI vocabulary mapping: translates universal platform terms to domain-appropriate language	JSON vocabulary map per locale
Role Definitions	Domain-specific RBAC role names and permission scopes	YAML role definition
Quality Gate Extensions	Additional quality gates specific to the domain (e.g. GMP batch quality gate)	Go quality gate plugin + YAML registration
Report Templates	Domain-specific compliance and operational report templates	Jinja2 templates + JSON schema

15.2 Available Domain Packs

Domain Pack	Version	Target Domains	Key Extensions
qala/financial-services	v3.1	Banking, Capital Markets, Investment Management	Financial Instrument and Investment Solution types; MiFID II CCR workflow; Bloomberg integration; backtesting quality gate; risk model validation
qala/pharmaceutical-gxp	v2.4	Drug Development, Clinical Trials, Manufacturing	Formulation and Clinical Trial solution types; GxP CCR workflow; FDA/EMA regulatory submission; LIMS integration; 21 CFR Part 11 e-signature
qala/cpg-manufacturing	v1.8	Consumer Goods, Food & Beverage, Personal Care	Product Line and Batch Record types; GMP compliance framework; SAP/ERP integration; EPCIS supply chain; REACH chemical compliance
qala/agritech	v1.3	Agriculture, Precision Farming, Agribusiness	Crop Management and IoT Firmware types; Nix edge SDE; FAOSTAT integration; field trial workflow; pesticide regulatory compliance

Domain Pack	Version	Target Domains	Key Extensions
qala/legal-regulatory	v2.0	Legal Services, Compliance, Regulatory Affairs	Regulatory Product and Legal Solution types; jurisdiction-aware release workflow; effective date scheduling; regulatory change impact analysis
qala/professional-services	v1.5	Consulting, Managed Services, Professional Services	Consulting Methodology and Service types; engagement lifecycle workflow; client delivery milestone integration; SOW version management
qala/research-academic	v1.2	Research Institutions, Universities, Pharma R&D	Research Asset and Experimental Framework types; dataset versioning; HPC compute integration; peer review workflow; replication evidence package
qala/software-enterprise	v4.0	Enterprise Software, SaaS, Platform Engineering	Enhanced Software types; SLSA Level 3 enforcement; enterprise SSO; SBOM generation; SOC 2 compliance evidence automation

15.3 Creating a Custom Domain Pack

Organisations can create custom Domain Packs to support proprietary solution types, internal processes, or industry-specific compliance frameworks. Custom Domain Packs go through a governance review before being activated in a factory.

1. Create a new Domain Pack directory with the standard structure: /schemas, /lifecycle, /workflows, /compliance, /integrations, /vocabulary
2. Define solution type schemas in YAML. Each schema must declare: type name, parent universal type, required fields, optional fields, and validation rules.
3. Define the lifecycle state machine overlay, specifying any additional states and their mapping to universal states.
4. Create the CCR workflow template, specifying approval roles and routing rules for each CCR risk level.
5. Package the Domain Pack as a versioned OCI artefact and publish to the Qala Marketplace or an internal registry.
6. Submit the Domain Pack for governance review (automated schema validation + manual review by platform team).
7. Once approved, activate the Domain Pack in the target factory via the Factory Settings panel.

16 UI/UX Design System

The Qala UI is built on a single design system with a persona-adaptive vocabulary layer. It serves 14 different persona types across all solution domains — from software engineers to pharmaceutical chemists to portfolio managers — while remaining one unified, coherent system.

16.1 Six Design Principles

Principle	Description	Implementation
Universal First, Domain-Native by Persona	Every core concept is universal. Vocabulary, iconography, and workflow shortcuts adapt per persona. A CPG chemist sees 'Batch Release Pipeline' — a developer sees 'CI/CD Pipeline' — identical underlying system.	Persona-adaptive vocabulary layer activated at login; adjustable in Profile Settings
Progressive Disclosure	The most common action for each persona is one click away. Advanced capabilities are discoverable without being visually dominant.	Context-sensitive sidebars; collapsible advanced panels; power user keyboard shortcuts
Trust Through Transparency	Every state change is visible. Every action produces an audit event. The UI makes governance legible — not bureaucratic.	Inline governance history; clear state indicators; audit event feed on every entity
AI as Companion	AI surfaces recommendations inline — dismissable, actionable, contextual. It never blocks a primary workflow.	Inline AI recommendation cards; one-click dismiss; recommendation reasoning accessible
Accessibility as Architecture	WCAG 2.1 AA minimum. Every interaction keyboard-navigable. Every colour combination meets contrast ratios.	Semantic HTML; ARIA labels; focus management; screen reader testing in CI
Speed as a Feature	Data tables load under 200ms. Command Palette returns results in < 100ms. Skeleton screens replace spinners.	Edge-cached responses; optimistic UI updates; virtual scrolling for large tables

16.2 Colour Palette

All colour decisions are token-first. No raw hex values appear in component code. Every colour is referenced by semantic token name, enabling theme switching and accessibility overrides without touching components.

Token	Hex	Usage	WCAG Ratio
brand-navy	#0D2B45	Primary headings, nav background, logo, brand anchors	AAA 18.1:1
action-blue	#1D6FA4	Interactive elements, links, primary buttons, progress indicators	AA 5.9:1
sky-light	#D6EAF8	Tints, info panels, selected state backgrounds, focus rings	—
surface-white	#FFFFFF	Primary content surface, card background, dialog background	—
surface-grey	#F4F6F8	Alternate row, secondary surface, sidebar track, divider fill	—
text-primary	#1A1A2E	Body text, labels, headings on white surface	AAA 18.1:1
text-muted	#64748B	Secondary labels, captions, metadata, placeholder text	AA 5.2:1
success-dark	#1A5C38	Gate passed, approved, active, deployed states	AA 10.1:1

Token	Hex	Usage	WCAG Ratio
warning-dark	#7D5A00	Medium risk, deprecated, awaiting review, pending states	AA 7.8:1
danger-dark	#7B241C	Error state, critical alert, failed gate, recall state	AA 9.1:1
ai-purple	#4A235A	AI Agent indicator, recommendation badge, intelligence features	AA 11.3:1
teal-dark	#0E6655	Domain extension, use-case headers, feature callouts	AA 8.7:1

16.3 Responsive Breakpoints

Breakpoint	Width	Primary Use	Layout
xs	< 640px	Field use — farm, lab, warehouse, delivery	Single column; bottom tab navigation; card-first
sm	640–767px	Large mobile, small tablet	Single column; sidebar as bottom drawer
md	768–1023px	Tablet — lab bench, meetings, client sites	Two column; collapsible sidebar overlay
lg	1024–1279px	Laptop — developer, analyst, consultant	Full sidebar; three-column content area; all table views
xl	1280–1535px	Desktop — compliance officer, operations centre	Expanded sidebar; multi-panel layout; Command Palette prominent
2xl	>= 1536px	Large monitor — analytics, monitoring centre	Ultra-wide multi-panel; side-by-side solution comparison; analytics dashboard

16.4 Core Component Library

The Qala design system includes the following core components. All components are available as React components with TypeScript props and Storybook documentation.

Component	Variants	Key Properties
SolutionCard	List view, Grid view, Compact	State badge, version indicator, factory tag, AI recommendation indicator, action menu
LifecycleTracker	Horizontal (default), Vertical (mobile)	Current state highlight, transition arrows, blocked state indicator, history popover
CCRPanel	Inline summary, Full detail drawer, Approval chain view	Risk score badge, approval chain progress, evidence list, comment thread
ArtifactTable	Full table, Compact list, Grid	Type icon, size, digest preview, download action, version comparison
VersionTree	Linear history, Branch view, Diff panel	Current version marker, branch indicators, diff line highlighting, rollback action
CommandPalette	Full-screen overlay (Cmd+K)	Global search, recent actions, entity navigation, AI-powered query suggestions
AIRecommendationCard	Inline banner, Side panel, Full modal	Confidence score, dismissal button, reasoning expandable, action button
AuditFeed	Timeline, Table, Export panel	Event type badge, actor attribution, timestamp, entity link, integrity verification

Component	Variants	Key Properties
StatCard	With trend, Without trend, Compact	Metric value, label, trend indicator, sparkline, drill-down link
GateStatusPanel	Gate list, Summary badge, Detailed view	Per-gate status, pass/fail badge, blocking indicator, evidence link

16.5 Persona-Adaptive Vocabulary

The Qala UI supports vocabulary adaptation per persona. The following table shows key vocabulary translations for three representative personas:

Universal Term	Software Developer	CPG Chemist	Portfolio Manager
Solution Development Environment	Development Environment	Lab Workspace	Research Workspace
Change Control Request (CCR)	Change Request	Formula Change Request	Model Change Request
Solution Lifecycle	Software Lifecycle	Product Lifecycle	Asset Lifecycle
Release Pipeline	CI/CD Pipeline	Batch Release Pipeline	Model Deployment Pipeline
Artefact	Build Artefact	Product Specification	Model Package
Quality Gate	Pipeline Gate	Batch Quality Check	Model Validation Gate
Solution Book	Documentation Site	Product Dossier	Investment Thesis
Solution Factory	Engineering Platform	Product Factory	Investment Platform

17 Observability & Operational Platform

Qala is observable by default. Every service emits structured logs, OpenTelemetry distributed traces, and Prometheus metrics. There is no operational state that cannot be observed without code changes. All observability data is queryable via Grafana dashboards and APIs.

17.1 The Three Pillars of Observability

Pillar	Tool	Scope	Retention
Metrics	Prometheus (scrape) + Grafana (visualisation)	All platform services; SDE resource utilisation; API request rates, latencies, and error rates; Kafka consumer lag; queue depths	13 months raw; 5 years downsampled
Traces	OpenTelemetry + Jaeger (storage) + Grafana Tempo	Distributed traces across all service-to-service calls; end-to-end request tracing from API gateway to database; AI Agent inference traces	30 days full; 12 months sampled (1%)
Logs	Loki (aggregation) + Grafana (visualisation)	Structured JSON logs from all services; SDE runtime logs; build pipeline logs; audit event export	90 days hot; 7 years cold (audit logs permanent)

17.2 SLO Dashboard

Qala's Grafana dashboards include a dedicated SLO dashboard that tracks platform-wide and per-tenant service level objectives in real time:

SLO	Target	Burn Rate Alert	Error Budget Window
API Availability	99.9% (Standard) / 99.95% (Enterprise)	Alert at 5x burn rate (1h window) and 2x burn rate (6h window)	30-day rolling window
API Read P99 Latency	< 80ms at 1,000 RPS/tenant	Alert at 90ms P99 for 5+ minutes	30-day rolling window
API Write P99 Latency	< 200ms for command acceptance	Alert at 250ms P99 for 5+ minutes	30-day rolling window
SDE Provision Time	< 10 minutes for software SDEs	Alert at 15-minute provision time	30-day rolling window
Event Processing Lag	< 500ms projection lag at median load	Alert at 2,000ms for 10+ minutes	7-day rolling window
Build Success Rate	> 95% of builds complete without infra errors	Alert at < 90% over 1-hour window	30-day rolling window

18 Hermetic Build & Supply Chain Security

Qala enforces hermetic, reproducible build environments as a first-class platform capability. Every build is fully isolated, deterministic, and tamper-evident. Supply chain security is governed through SLSA compliance and cryptographic attestation.

18.1 Hermetic Build Principles

Principle	Description	Enforcement Mechanism
Full Isolation	Build containers have no access to host network, filesystem, or ambient credentials. All inputs must be declared explicitly.	Kubernetes network policy: no egress from build pods; read-only root filesystem; no ambient service account tokens
Immutability	Once a build environment is defined and locked, no dependency can change without a version bump and CCR.	Digest-pinned dependencies in SDE manifest; change requires CCR approval; manifest version enforced at build time
Environment-as-Code	All build environments are defined in version-controlled configuration files. The definition is the environment.	SDE manifest (.qala.yaml) in version control; no manual environment changes permitted without manifest update
Frozen Dependencies	All dependencies — OS packages, language libraries, tools — are pinned to exact versions with cryptographic digests.	No 'latest' references permitted in manifests; SHA-256 digest verification before any dependency is used
Reproducibility	Given the same source commit and environment definition, any build must produce bit-for-bit identical outputs.	Hermetic build container; deterministic build toolchain; reproducible build flags enforced in toolchain configuration

18.2 SLSA Compliance Levels

SLSA Level	Requirements	Qala Implementation
Level 1	Build process is documented; artefacts have provenance	All builds produce a provenance document; Build Service records every build invocation
Level 2	Hosted build service; signed provenance; version-controlled source	Build Service is a hosted service; provenance signed with platform key; all source in version-controlled repositories
Level 3	Hardened build platform; hermetic builds; non-forgeable provenance; audit logs retained	Full hermetic isolation; build pods have no network egress; provenance signed by isolated build agent key; audit logs in immutable vault
Level 4 (target)	Two-party review; hermetic builds; pinned dependencies; retention policy	CCR requirement for environment changes (two-party review); hermetic builds (Level 3 compliance achieved); pinned deps enforced; 7-year retention

18.3 Software Bill of Materials (SBOM)

Qala automatically generates a Software Bill of Materials (SBOM) for every software build. The SBOM is attached to the artefact registry entry and is required for release gate passage in Enterprise and regulated deployments.

SBOM Property	Description
Format	CycloneDX 1.5 (primary); SPDX 2.3 (secondary, on request); both formats available via API
Scope	All direct and transitive dependencies; OS-level packages; container base image components

SBOM Property	Description
Freshness	Generated on every build; previous SBOMs retained with the associated artefact version
Vulnerability Correlation	SBOM is continuously scanned against NVD, OSV, and vendor advisory databases; new CVEs trigger alerts even for old artefact versions
Distribution	Available to authorised consumers via pre-signed URL or embedded in release bundle; signed with platform key
Regulatory Use	SBOM can be submitted as part of regulatory evidence packages for FDA, EMA, or other regulatory authority submissions

19 Quality Management System

Qala's Quality Management System (QMS) provides universal test management, defect tracking, quality gate evaluation, and benchmarking across all solution types. The QMS is designed to support both automated software testing and domain-specific quality validation workflows.

19.1 Test Management

Test Type	Universal	Automated Execution	Domain-Specific Variants
Validation Tests	Yes	Yes (where applicable)	Software: system / acceptance tests; Pharma: clinical validation; Financial: model validation; Research: hypothesis testing
Verification Tests	Yes	Yes	Software: unit / integration tests; Physical Goods: specification verification; Financial: parameter verification
Compliance Tests	Yes	Partial (automated evidence collection)	ISO 27001 controls; GxP compliance checks; MiFID II requirements; SOC 2 controls
Performance Tests	Yes	Yes	Software: load / stress / soak tests; Financial: backtesting; Research: computation performance
Regression Tests	Yes	Yes	Software: automated regression suite; Formulation: comparative study; Financial: historical performance comparison
Security Tests	Yes (software focus)	Yes	SAST; DAST; dependency scanning; container image scanning; penetration testing (scheduled)

19.2 Defect Management Lifecycle

State	Description	Owner	SLA by Severity
New	Defect created from test failure, security scan, or manual report	Reporter	Critical: 1h triage; High: 4h; Medium: 24h; Low: 72h
Triaged	Defect reviewed, severity/priority confirmed, assigned to developer	Team Lead / Manager	—
In Progress	Developer actively working on root cause and fix	Assignee	Critical: 24h fix; High: 72h; Medium: 1 sprint; Low: backlog
Fixed	Fix implemented; awaiting verification by test or reviewer	Assignee	—
Verified	Fix confirmed by test execution; regression tests passing	QA / Test Engineer	—
Closed	Defect fully resolved; release can proceed	Team Lead	—
Deferred	Defect acknowledged; resolution deferred to future version with risk acceptance	Product Manager / Manager	Risk acceptance required for High/Critical deferrals

State	Description	Owner	SLA by Severity
Won't Fix	Defect will not be resolved (by design, out of scope, or accepted risk)	Product Manager + Factory Admin	Requires documented approval for Critical/High severity

19.3 Quality Benchmarking

Qala provides cross-solution and cross-factory quality benchmarking — allowing organisations to compare their solutions against industry benchmarks and historical performance.

Benchmark Metric	Scope	Collection Method	Visualisation
Defect Density	Defects per 1,000 lines of code / per component	Automatic from defect records + code metrics	Trend chart; factory percentile ranking
Test Coverage	% of solution under test by type (unit, integration, e2e)	Automatic from test run results	Coverage gauge; comparison to factory target
Change Failure Rate	% of releases that require rollback or hotfix	Automatic from release and rollback events	Trend chart; industry comparison (where available)
Lead Time for Change	Time from commit to production deployment (mean)	Automatic from solution event timestamps	Histogram; factory trend; benchmark comparison
Mean Time to Recovery (MTTR)	Mean time from incident to restored service	Manual incident records + automated recovery detection	Scatter plot; trend; industry benchmark
CCR Approval Cycle Time	Mean time from CCR submission to approved state	Automatic from CCR event timestamps	Trend chart; breakdown by CCR type and risk level

20 Open Design Decisions (RFC Index)

Qala maintains a formal Request for Comments (RFC) process for significant architectural and design decisions. This chapter lists all active RFCs and their current status. Each RFC has a 30-day comment period before the Architecture Review Board issues a decision.

20.1 RFC Index

RFC ID	Title	Domain	Status	Decision By
RFC-001	Polyglot Architecture and Language Selection	Architecture	Open — Decision Pending	Pre-alpha build
RFC-002	SDE Model: Universal Environment Types	Platform	Open — Decision Pending	Pre-alpha build
RFC-003	Hermetic Build and Attestation for All Solution Types	Build / Security	Open — Decision Pending	Pre-alpha build
RFC-004	Universal Event Architecture and Kafka Topology	Platform	Decision Made — Kafka 3.7 KRaft	Completed
RFC-005	AI Agent: Universal Intelligence Architecture	AI / Intelligence	Open — Decision Pending	Pre-beta
RFC-006	Universal Solution Type System and Extensibility	Solution Model	Decision Made — YAML DSL Domain Packs	Completed
RFC-007	Change Control Governance: Universal CCR Model	Governance	Decision Made — Risk-based routing	Completed
RFC-008	Universal Quality Management Model	Quality	Open — Decision Pending	Pre-beta
RFC-009	Universal Release and Distribution Management	Release	Open — Decision Pending	Pre-beta
RFC-010	Physical Goods, CPG and Manufacturing Domain Extension	Domain Extension	Open — Comment Period Active	Pre-GA
RFC-011	Financial, Investment and Capital Solution Domain Extension	Domain Extension	Open — Comment Period Active	Pre-GA
RFC-012	Agricultural and Agribusiness Solution Domain Extension	Domain Extension	Open — Comment Period Active	Pre-GA
RFC-013	Tools, Toolchains, Libraries, Packages and SDK Domain Extension	Domain Extension	Decision Made — Toolchain registry model	Completed
RFC-014	Service, Consulting and Professional Services Domain Extension	Domain Extension	Open — Comment Period Active	Pre-GA
RFC-015	Business, Process and Operational Solution Domain Extension	Domain Extension	Open — Comment Period Active	Pre-GA
RFC-016	Research, Academic and Scientific Solution Domain Extension	Domain Extension	Open — Comment Period Active	Pre-GA
RFC-017	Legal, Tax and Regulatory Solution Domain Extension	Domain Extension	Open — Comment Period Active	Pre-GA

RFC ID	Title	Domain	Status	Decision By
RFC-018	Universal Compliance Framework Architecture	Compliance	Decision Made — Plugin-based framework adapters	Completed
RFC-019	Physical World Integration: ERP, MES, LIMS, IoT Bridges	Integration	Open — Decision Pending	Pre-GA
RFC-020	Multi-Tenancy, Tenant Isolation and Cross-Tenant Sharing	Platform	Decision Made — RLS + namespace isolation	Completed
RFC-021	Identity, RBAC and Electronic Signature Architecture	Security	Open — Decision Pending	Pre-beta
RFC-022	Open Source, Ecosystem and Commercialisation Strategy	Strategy	Open — Decision Pending	Pre-GA

20.2 Key Open Design Questions

Q-ID	Question	Implications if Deferred
Q-ARCH-001	Is Scala/Akka worth the language diversity cost for streaming pipelines, or can Go + Temporal deliver equivalent capability?	If deferred past pre-alpha: streaming pipeline architecture may be difficult to change post-implementation
Q-ARCH-002	Is Rust's memory safety genuinely necessary for the Kernel Plane, or is Go's improved GC sufficient?	Language choice affects hiring, toolchain, and security posture for the highest-criticality platform code
Q-SDE-001	Should 'SDE' be a unified concept with modular capability packs, or explicitly subtyped for different domains?	Subtyped SDEs are more intuitive for non-software users; unified model is simpler to govern — decision affects v1 API contract
Q-SDE-004	Should non-software SDE types be in scope for v1, or shipped in subsequent releases via Domain Packs?	Phasing reduces v1 scope risk; delays non-software market entry and differentiation from software-only competitors
Q-HB-002	For stochastic processes (ML, financial modelling), should attestation require 'governed reproducibility' within tolerance bounds rather than bit-for-bit reproducibility?	Without clear standard, regulated users cannot use platform for ML/financial model governance
Q-AI-001	Should the AI Agent be opt-in or opt-out by default? Opt-in respects privacy preferences; opt-out maximises early user value delivery.	Affects initial activation rate and privacy policy compliance across jurisdictions

21 Glossary

This glossary defines all key terms used throughout this document. Terms are listed alphabetically.

Term	Definition
ADR (Architecture Decision Record)	A document capturing a significant architectural decision: the context, the options considered, the decision made, and the consequences. Stored in the Solution Playbook.
Aggregate	In event sourcing, a cluster of domain objects that is treated as a single unit of consistency. Aggregates protect invariants and emit domain events in response to commands.
Artefact	Any versioned, immutable output produced by an SDE: compiled binaries, packaged releases, test reports, compliance evidence documents, and attestation records.
Attestation	A cryptographically signed record proving that an artefact was produced from specific, verified inputs in a governed environment. Qala generates SLSA-compatible attestations for all builds.
CCR (Change Control Request)	A formal request to make a governed change to a solution, SDE configuration, or platform entity. CCRs are risk-scored, routed for approval, and tracked to implementation.
CQRS	Command Query Responsibility Segregation — an architectural pattern where the read (query) and write (command) paths for a service are separated. Write path updates the event store; read path uses materialised projections.
Domain Pack	A versioned extension package that adds domain-specific capabilities to the Qala platform: solution type schemas, lifecycle overlays, CCR workflow templates, compliance framework adapters, and integration bridge configurations.
Event Sourcing	A persistence strategy where the state of a domain entity is stored as an ordered log of immutable events, rather than as a current-state record. Current state is derived by replaying events.
Factory Blueprint	A packaged, versioned configuration of a Solution Factory — including SDE templates, domain packs, toolchains, and governance policies — that can be shared through the Marketplace and instantiated by others.
Hermetic Build	A build process that is fully isolated from its environment: no ambient credentials, no external network access, all inputs explicitly declared and digest-pinned. Hermetic builds are reproducible and tamper-evident.
Lifecycle State Machine	The platform-enforced graph of valid states and transitions for a domain entity (solution, SDE, CCR, release). Invalid transitions are rejected by the platform.
Playbook	The design and decision record for a solution: architecture decision records, design documents, risk register, stakeholder map, change history, and quality benchmarks. Versioned alongside the solution.
Projection	A read-optimised materialised view of a domain entity's current state, derived by replaying domain events. Projections are eventually consistent with the event store.
RBAC (Role-Based Access Control)	An access control model where permissions are grouped into roles, and roles are assigned to users. Qala uses hierarchical RBAC with roles defined at the factory level and assignable at factory, SDE, or solution scope.
Release Gate	An automated or manual check that must pass before a release can progress to the next stage. Quality gates collectively form a release gate that governs promotion from development to production.
SDE (Solution Development Environment)	The foundational deployable unit of Qala — a fully self-contained, versioned, and governable workspace that defines all conditions for creating and operating solutions. The SDE is the kernel of the Qala operating system.
SEM (Security Event Monitor)	A dedicated Qala platform service in the Intelligence Plane providing continuous threat detection, vulnerability scanning, and automated security response across all SDEs and solutions.
SLSA (Supply-chain Levels for Software Artefacts)	An open security framework providing a checklist of standards and controls to prevent artefact tampering and improve supply chain security integrity. Qala targets SLSA Level 3 for all software builds.

Term	Definition
SBOM (Software Bill of Materials)	A formal, machine-readable inventory of all components used to build a software product — including direct and transitive dependencies, versions, and cryptographic digests.
Solution	Any purposeful output designed to address a problem, fulfil a goal, or produce an intended outcome. In Qala, solutions span every domain and scale, from personal projects to enterprise systems to physical goods.
Solution Book	The comprehensive documentation repository for a solution: API reference, configuration guide, runbooks, release notes, training materials, and compliance evidence. Auto-generated and team-authored. Versioned with the solution.
Solution Factory	The primary organisational unit in Qala — a governed namespace that produces and manages Solution Development Environments, child factories, and all solution activity within a defined scope.
Solution Model	A schema and blueprint that defines the structure, required fields, lifecycle states, and governance rules for a category of solutions. Solutions are instances of a Solution Model.
Toolchain	A hierarchically organised collection of tools used within an SDE: Toolkits (domain-level bundles) → Toolsets (functional groups) → Tools (individual tools). The toolchain is versioned and governed.
Vault (HashiCorp Vault)	The secrets management and PKI system used by Qala for: per-tenant encryption key storage, dynamic secret generation, automatic credential rotation, and mTLS certificate issuance for all platform services.
Zero-Trust	A security model that assumes no implicit trust between services or users, regardless of network location. Every request must be authenticated and authorised. mTLS is enforced for all service-to-service communication in Qala.

22 Appendices

Appendix A — Document Revision History

Version	Date	Author	Changes
0.1	2025-Q3	Platform Engineering	Initial draft — Platform Concept and SDD v1 synthesis
0.2	2025-Q4	Platform Engineering + Architecture	Added SDD v2, LLD v1, RFC v2 content; expanded data model chapter
0.3	2026-Q1	Platform Engineering	Added Requirements v2, Workflows v2, UI/UX Spec v1; restructured chapter layout
0.9	2026-Feb	Architecture Review Board	Pre-final review; added Observability, SBOM, Quality chapters; resolved RFC-004, RFC-006, RFC-007
1.0	2026-Mar	Architecture Review Board + Product	Final document — consolidated from all 16 source documents; approved for distribution

Appendix B — Source Document Index

Document ID	Title	Version	Key Contributions to This Document
QALA-CONCEPT-v1	Platform Concept & Architecture	v1.0	Entity ontology, hierarchy model, domain pack schema, SDE manifest format, factory type definitions
QALA-SDD-v1	Software Design Document	v1.0	Initial service decomposition, API surface design, initial data model
QALA-SDD-v2	Extended Software Design Document	v2.0	Hermetic build architecture, SDE composition, SDE backup/recovery, SLSA compliance, build attestation
QALA-LLD-v1	Low-Level Technical Design Document	v1.0	Aggregate boundaries, canonical event schemas, PostgreSQL schema, Kafka topic architecture, NFR targets
QALA-RFC-v2	Request for Comments — Universal Edition	v2.0	22 RFCs covering architecture, domain extensions, compliance, AI, and strategy; six-plane architecture model
QALA-PRD-v1	Product Requirements Document	v1.0	Initial feature definitions, user stories, acceptance criteria
QALA-PRD-v2	Product Requirements Document — Universal Edition	v2.0	Domain-specific feature requirements, 14 persona definitions, domain pack PRD requirements
QALA-REQ-v1	Requirements Specification	v1.0	Initial functional and non-functional requirements baseline
QALA-REQ-v2	Requirements Specification — Universal Edition	v2.0	65 use cases; MoSCoW-prioritised functional requirements across all ten capability pillars and all domains
QALA-WF-v2	Workflows & Use Cases — Universal Edition	v2.0	65 use cases across 11 domain chapters; workflow state machines; CCR routing logic; release pipeline definitions
QALA-UIUX-v1	UI/UX Specification, Design System & User Flows	v1.0	Design system (colours, typography, components); 14 persona specifications; 12 user flows; 35 screen specs
QALA-UNIFIED-v1	Unified Design & Architecture Document	v1.0	Cross-cutting synthesis; seven core problems; binding principles table; entity hierarchy refinement
QALA-BIZ-v1	Business Plan	v1.0	Market sizing, competitive landscape, commercialisation strategy, pricing model

Document ID	Title	Version	Key Contributions to This Document
QALA-HIFI-v1	Hi-Fidelity Screen Specifications	v1.0	Screen layout specifications for 25 primary UI screens; component-level annotations
QALA-IND-WF-v1	Industry Workflows	v1.0	Cross-industry workflow patterns; domain-specific lifecycle adaptations
QALA-WF-UC-v1	Workflows Use Cases	v1.0	Detailed use case specifications with step-by-step actor-system interaction tables

Appendix C — Requirements Traceability

The following table maps key architectural decisions in this document to the requirements they satisfy:

Requirement ID	Requirement Summary	Design Chapter	Design Decision
FR-SM-UNI-001	Universal solution type taxonomy across ten categories	Chapter 3	Six first-class solution types + Domain Pack subtypes
FR-SDE-UNI-001	SDE provisioning for all supported solution type categories	Chapter 5	Modular SDE types with domain-specific capability packs
FR-SDE-SW-001	Hermetic builds for software SDEs	Chapter 18	OCI container + Kubernetes; digest-pinned dependencies; SLSA Level 3
FR-CM-UNI-001	All configuration artefacts versioned	Chapter 5.5	SDE manifest (.qala.yaml) version-controlled; all changes logged
FR-CCR-UNI-001	CCR workflow for all governed changes	Chapter 7	Universal CCR data model with domain-specific approval chain overlays
FR-CCR-UNI-003	Risk score computation for every CCR	Chapter 7.4	Weighted risk scoring algorithm across six factors
FR-TEST-UNI-001	Test Management Service for all solution types	Chapter 19	Universal QMS with domain-specific test type extensions via Domain Packs
FR-BUG-UNI-001	Defect tracking for all solution types	Chapter 19.2	Universal defect lifecycle state machine with severity-based SLAs
NFR-PERF-001	API P99 latency < 80ms (read)	Chapter 8.4	CQRS + read projections in Redis; CDN caching for static content
NFR-SEC-001	Zero-trust security model	Chapter 10	Istio mTLS; per-request JWT validation; no implicit trust
NFR-COMP-001	Immutable audit trail from day one	Chapter 9	Append-only audit_events table; Kafka fan-out projection; immutable Vault storage

Appendix D — API Endpoint Reference (Summary)

A summary of the primary REST API endpoints. Full OpenAPI specification is available at /api/v1/openapi.json in any running Qala instance.

Endpoint	Method	Description	Auth Required
/api/v1/factories	GET, POST	List or create Solution Factories	Bearer JWT; factory:read or factory:create
/api/v1/factories/{id}	GET, PATCH, DELETE	Get, update, or archive a Solution Factory	Bearer JWT; factory:read or factory:update

Endpoint	Method	Description	Auth Required
/api/v1/factories/{id}/sdes	GET, POST	List or create SDEs within a factory	Bearer JWT; sde:read or sde:create
/api/v1/sdes/{id}	GET, PATCH	Get or update an SDE configuration	Bearer JWT; sde:read or sde:update
/api/v1/sdes/{id}/lifecycle	POST	Trigger SDE lifecycle transition (activate, suspend, archive)	Bearer JWT; Factory Admin role
/api/v1/solutions	GET, POST	List or create solutions (filtered by factory/SDE)	Bearer JWT; solution:read or solution:create
/api/v1/solutions/{id}	GET, PATCH	Get or update a solution	Bearer JWT; solution:read or solution:update
/api/v1/solutions/{id}/lifecycle	POST	Trigger solution lifecycle state transition	Bearer JWT; solution:transition
/api/v1/ccrs	GET, POST	List or create Change Control Requests	Bearer JWT; ccr:read or ccr:submit
/api/v1/ccrs/{id}/approve	POST	Approve a CCR (advances approval chain)	Bearer JWT; ccr:approve + Approver role
/api/v1/releases	GET, POST	List or create release records	Bearer JWT; solution:release
/api/v1/releases/{id}/publish	POST	Publish a release to distribution targets	Bearer JWT; Release Manager role
/api/v1/artefacts/{id}	GET	Get artefact metadata and download URL	Bearer JWT; artefact:read
/api/v1/audit/events	GET	Query audit events with filters	Bearer JWT; audit:read
/api/v1/search	GET	Full-text and semantic search across all entities	Bearer JWT
/api/v1/ai/recommendations	GET	Get AI recommendations for a specified entity	Bearer JWT

Appendix E — Compliance Framework Coverage

Framework	Coverage Level	Evidence Generated	Domain Pack Required
ISO 27001	Full	ISMS controls evidence; access control audit; incident log; change management records	No (platform baseline)
SOC 2 Type II	Full	Availability, confidentiality, and security control evidence; audit event export	No (platform baseline)
GDPR / Data Privacy	Partial (data processing records)	Data processing records; consent audit; right-to-erasure workflow	No (platform baseline)
21 CFR Part 11 (FDA)	Full	Electronic signature audit trail; audit log integrity; system validation evidence	qala/pharmaceutical-gxp
EU Annex 11 (EMA)	Full	GxP validation documentation; computerised system risk assessment; backup evidence	qala/pharmaceutical-gxp
MiFID II	Partial	Model change governance records; CCR audit trail for model changes; transaction reporting linkage	qala/financial-services
GxP (general)	Full	Batch record linkage; change control records; qualification documentation	qala/cpg-manufacturing

Framework	Coverage Level	Evidence Generated	Domain Pack Required
SLSA Level 3	Full	Build provenance; hermetic build evidence; artefact attestation records	No (platform baseline for software)
FedRAMP (target)	In Progress	Government cloud deployment configuration; security control evidence mapping	Planned — qala/government-cloud

— End of Document —

Qala Platform Design & Architecture Document · v1.0 · March 2026 · Confidential